Munich University of Applied Sciences
HM Business School

From Vibe Coding to Production-Ready Software

Developing and Evaluating a Governance and Development Framework for Generative AI-Assisted Software Creation by Non-Technical Employees

Master's Thesis

Submitted by:
Nassib Aboud

Degree Program:
Business Information Systems (M.Sc.)

Matriculation Number:
34140924

Contact:
nassib.aboud@hm.edu

First Examiner:
Prof. Dr. Markus Thimmel

Second Examiner:
Prof. Dr. Christoph Ertl

Submission Date:
30 October 2026

Declaration on the Use of Generative AI

Generative artificial intelligence tools were used as supporting tools during the preparation of this Master's thesis.

Their use included, where applicable, support with literature discovery, brainstorming and structuring, language and style improvement, critical review of written material, and assistance with software development and coding tasks conducted as part of the research.

Generative AI outputs were not treated as independent academic sources. Academic claims and evidence were verified against the original sources where applicable.

AI-generated suggestions, analyses, text, and code were critically reviewed, selected, modified, or rejected by the author.

The author remains fully responsible for the research design, methodological decisions, interpretation of results, argumentation, conclusions, and final content of this thesis.

The use of generative AI tools was conducted in accordance with the applicable requirements of Munich University of Applied Sciences and the principles of good scientific practice.

Declaration of Originality

I hereby declare that I have prepared this Master's thesis independently and that I have used only the sources, materials, and tools indicated in the thesis.

All passages that have been taken directly or indirectly from published or unpublished sources have been identified accordingly.

The use of generative artificial intelligence and other digital tools has been disclosed where required.

I further declare that this thesis has not previously been submitted, in the same or substantially similar form, as part of another examination procedure.

Munich, 6 September 2026

Nassib Aboud

Abstract

Insert abstract here.

Contents

Abstract **3**

1 Introduction **6**

1.1 Research Objectives and Research Questions 6

1.2 Scope and Delimitations 6

2 Research Methodology REVIEWED - Application Evidence, data analysis and evidence integatin as well as research quality all need to look over again **7**

2.1 Research Design 7

2.2 Research Context, Roles and Artifact 8

2.3 Research Objectives and Phases 9

2.4 Evaluation Criteria 10

2.5 Application Evidence 11

2.6 Practitioner Interviews 12

2.7 Data Analysis and Evidence Integration 13

2.8 Research Quality and Limitations 13

3 Problem and framwork requirments **14**

4 Theoretical Background and Related Research **14**

4.1 Conceptual Foundations and Key Definitions 14

4.2 Software development before and after GenAI REVIEWED 16

4.3 Capabilities and Limitations of Non-Technical Developers 19

4.4 Expert Support in Citizen-Led Software Development 19

4.5 Governance, Guardrails, and Development Support 19

4.6 Conceptualisation of Production Readiness 19

4.7 Synthesis of the Literature and Research Gap 19

5 The Governance and Development Framework **19**

5.1 The Enabling Environment 19

5.1.1 Technology Platform 19

5.1.1.1 Security rules and checks 19

5.1.2 Governance Structure 20

5.2 Working Practices 20

5.2.1 Core Concepts and Principles 20

5.2.2 The Stage-Gate Process Foundation REVIEWED 20

5.2.2.1 Overall structure 21

5.2.2.2 Gate decisions 21

5.2.2.3 Risk-based decisions 22

5.2.2.4 Roles and responsibilities 22

5.2.3 The Stages of the Stage-Gate Process REVIEWED 23
5.2.3.1 Screen and scope 23
5.2.3.1.1 Problem and business case 23
5.2.3.1.2 Suitability for citizen development 23
5.2.3.1.3 Reuse and technical complexity 24
5.2.3.1.4 Data, risk and legal requirements 24
5.2.3.1.5 Capability of the citizen developer 24
5.2.3.1.6 Infrastructure 25
5.2.3.1.7 Experiment boundaries and gate G1 25
5.2.3.2 Prototype 26
5.2.3.3 Build and verify 28
5.2.3.3.1 General Best Practices 28
5.2.3.3.2 The Agile V Development Framework 29
5.2.3.3.3 Left side: specifying the change 31
5.2.3.3.4 Bottom: building the change 33
5.2.3.3.5 Right side: verifying and accepting the change 36
5.2.3.4 Test application 40
5.2.3.5 Pilot 45
5.2.3.6 Operate and reassess 45

6 Evaluation and Findings **45**
6.1 Starting Point 46

7 Discussion **46**

8 Conclusion **46**

A Testing across the development levels **49**

1 Introduction

Generative AI allows people to build software by describing what they want in plain language. Employees without programming training can now create working prototypes within hours or days (Kobiella et al., 2026). Many of these prototypes, however, never become software that an organisation can safely rely on. Maes (2026) calls this a deployment paradox. The difficult part is no longer writing code. It is making sure that the software is secure, legally compliant, reliable, useful for the business and clearly owned (Farrag, 2026; Kobiella et al., 2026). Without guidance, applications built outside IT can become shadow IT and technical debt that the organisation later has to maintain (Viljoen et al., 2024).

This thesis addresses this problem in the HR department of SIXT. Since June 2026, the researcher and the researcher's manager have built three HR applications with generative AI. Their experience, together with the literature, forms the basis of a framework that guides non-technical employees from an idea to software that can be used in daily work.

1.1 Research Objectives and Research Questions

The thesis answers one main research question:

> **RQ:** How can a governance and development framework enable non-technical employees to develop software with generative AI that is ready for production use in a large organisation?

The objective of this thesis is to develop and evaluate a governance and development framework for software that non-technical employees create with generative AI. The framework should help them move from a prototype to production use, keep risks under control and involve experts where needed.

Four sub-questions divide the main question into smaller parts:

SQ1 Which difficulties arise when non-technical employees use generative AI to build software for production use, and where do they need support?

SQ2 Which working practices, safeguards and organisational conditions should a framework include to address these difficulties?

SQ3 Does the framework address these difficulties in practice, and how useful and feasible do practitioners consider it?

SQ4 Which organisational conditions does the framework require, and what limits its transfer to other settings?

The study answers these questions in three phases. The first phase examines the starting point with the first application and answers SQ1 (Section 6.1). The second phase develops the framework and answers SQ2 (Section 5). The third phase evaluates the framework with the third application and practitioner interviews and answers SQ3 and SQ4 (Section 6).

1.2 Scope and Delimitations

This thesis examines software that is developed primarily by non-technical employees who use generative AI coding tools, such as Claude Code, as their main means of development. The defining characteristic is therefore the way in which the software is created, not whether the resulting application itself contains AI functionality. Following this focus, non-technical employees, referred to here as citizen developers, are defined as employees who neither hold a professional software engineering role nor have formal training in software development, although they may possess general digital skills. [Add one sentence on how the development team fits this definition, e.g. that the researcher studies Business Information Systems but had no professional programming experience before the study.]

The study is situated in the HR department of SIXT. The applications examined are internal tools that display, edit or track HR data for a limited group of internal users. Their development was supported by HR IT and, where necessary, by central IT, within the resources these units could provide.

Several related areas lie outside the scope of this thesis. Traditional low-code and no-code platforms, which do not rely on AI code generation, are considered only for comparison. Likewise, the thesis does not address professional software engineers who use AI to accelerate their own work, since its focus lies on employees who depend on expert support. Risks arising from AI features within an application, such as harmful chatbot responses, are also excluded, as these are addressed by dedicated methods such as MeRGE (Fetzer et al., 2026). Furthermore, the framework does not replace formal legal, security or data protection assessments; instead, it identifies the points at which such specialist expertise is required. Finally, business value is considered only as part of the first gate of the framework, and no separate model for its calculation is developed.

Importantly, the framework does not presuppose that every idea should reach production. Stopping an idea, restricting its scope or handing it over to IT are regarded as legitimate outcomes, since preventing unsuitable applications from advancing is itself part of responsible governance.

2 Research Methodology REVIEWED - Application Evidence, data analysis and evidence integatin as well as research quality all need to look over again

2.1 Research Design

The study follows a design science research (DSR) approach. DSR creates and evaluates artifacts that solve practical problems. In this thesis, the artifact is the governance and development framework. Following Sein et al. (2011), it is an *ensemble artifact*: it combines technical elements, such as approved tools and sandboxes, with organisational elements, such as roles, gates and working practices.

DSR fits this thesis because its aim is not only to describe a problem but to develop a usable solution and assess it under real organisational constraints. The literature provides foundations for the design. It cannot show on its own, however, whether citizen developers can apply the framework with limited IT support. This requires evidence from the setting in which the framework is meant to be used.

Peffers et al. (2007) describe DSR as a process of problem identification, definition of solution objectives, design and development, demonstration, evaluation and communication. The study is also informed by action design research (ADR), which treats building an artifact, intervening in an organisation and evaluating the artifact as interwoven activities rather than separate steps (Sein et al., 2011). Sein et al. (2011) criticise sequential process models such as that of Peffers et al. (2007) for separating building from evaluation. This critique applies here: the framework was developed while software was being built at SIXT, and practical experience changed it along the way. The process of Peffers et al. (2007) is therefore used to structure how the research is reported, not to claim that all activities happened in a strict order.

ADR served as methodological guidance; the study did not follow every stage and principle of a formal ADR process. The study is instead reported in three phases (Table 1). These phases are not complete research cycles. Phase 1 assesses the starting point before any version of the framework existed. Phase 2 partly applies an earlier version of the framework and revises it. Phase 3 applies and evaluates the final version, but its findings are not built into a further version within this thesis.

Table 1. The three phases of the study (own representation).

Phase	Purpose	Main evidence	SQ	DSR activity
1 Starting point	Identify the difficulties of developing without a framework	App 1: development records, user feedback, IT review; formative discussions	SQ1	Problem identification
2 Framework development	Design practices and conditions that address these difficulties	Phase 1 findings, literature, formative discussions, trial of selected practices in App 2	SQ2	Objectives, design, demonstration
3 Evaluation	Assess the framework as a whole	App 3: records, tests, user feedback, IT review; final interviews	SQ3, SQ4	Evaluation

The evaluation within the thesis was both formative and summative (Sein et al., 2011; Venable et al., 2016). Formative evaluation takes place while an artifact is still being developed and aims to improve it. Summative evaluation takes place once the artifact is finished and aims to judge its value. In Phase 2, experience and feedback from applying an earlier version shaped the framework (formative evaluation). In Phase 3, the final version is evaluated as a whole (summative evaluation). The evaluation section documents the summative evaluation, while the formative evaluation is reported less systematically at several points in this thesis. A controlled experiment was not feasible. Such an experiment would compare groups with and without the framework under identical conditions, so that any difference could be attributed to the framework alone. Sein et al. (2011) argue that, in organisational design research, an authentic setting, meaning real use in a real organisation, matters more than a controlled one. The three applications therefore offer a natural comparison: one application was built without the framework, one with parts of it and one with the final version. This comparison is uncontrolled, because other factors, such as the team's growing experience or the quality of the organisational adn technical enviroment or the coding models, also changed between the applications.

The design decisions of the framework drew on several sources: the researcher's own experience while building the applications, feedback from users and citizen developers, scientific literature and practitioner sources. Practitioner sources were adopted when they proved useful in practice and were perceived positively in informal feedback from citizen developers. The exact origin of each design decision was not recorded. Section 5 therefore presents the final combination of decisions, together with the practical arguments for each and the scientific or practitioner sources behind it.

2.2 Research Context, Roles and Artifact

The study takes place within SIXT HR. The researcher and their manager acted as citizen developers. Both began without professional programming skills and acquired knowledge through practical development, training and support from IT. The researcher held two roles: designer of the framework and citizen developer. The manager acted occasionally as a co-citizen developer and mainly as the business owner. HR IT and, occasionally, central IT provided support within their available resources. The existing corporate environment evolved throughout the process as requested by the researcher, although certain limitations remained. It will be explained in more detail later on. Therefore, the thesis distinguishes this existing infrastructure from the practices it proposes.

Three HR applications provide the practical setting for the study. They address different business tasks but share a focus on making structured HR information easier to view, update and track.

> **Organisation explorer:** The organisation explorer presents organisational structures and related HR information through a visual interface. It enables users to explore the organisation and access information according to their assigned roles. The application combines data presentation with dashboard functionality to make organisational information easier to navigate. It was the team's first application and was developed without the thesis framework. User feedback revealed that some interface choices and features were unnecessarily complicated, leading to substantial rework.

Reorganisation tool: The reorganisation tool supports working with organisational data in the context of structural changes. It provides a visual interface for reviewing information, making changes and tracking relevant data. Different access roles determine which functions and information users can access. The application also includes a chatbot that helps users obtain information. With users' consent, chatbot conversations were reviewed to identify incorrect responses and opportunities for improvement. Compared with the first application, development included an initial prototype and checks for existing libraries before implementing functionality.

HR job-profile tool: The HR job-profile tool supports creating, viewing and maintaining structured job-profile information. It provides an interface for working with the underlying HR data, with access determined by user roles. The application serves as the principal case for applying the combined framework, including its enabling environment and working practices. Development incorporated training, Git version control, prototyping, library checks and development records such as `plan.md`. The evaluation examines the actual coverage of these practices and the exceptions encountered during their use.

Across the three applications, the main functions involve visualising, editing and tracking HR data. Their different business purposes provide opportunities to examine development challenges across related use cases within the same organisational setting.

The organisational environment included approved tools, sandboxing, deployment infrastructure, password management and processes for requesting data and technical assistance. Their availability and application varied during the study. These organisational and technical conditions are described in the framework chapter.

The framework constitutes the research artifact and consists of two connected parts. The enabling environment describes the organisational and technical conditions needed to support citizen development. The working practices describe development activities, responsibilities, evidence requirements, practices for AI-assisted development and decision gates. Existing organisational enviroment and technical infrastructure is distinguished from practices proposed or adapted by the thesis.

The study period covers approximately June to November 2026. During this period, the framework developed through informal iterations. Two main versions are distinguished. The first focused primarily on working practices. The second retained most of these practices and added the organisational and technical conditions needed to apply them. The second version is presented as the final artifact and forms the basis of the final practitioner evaluation. Earlier development experiences and formative discussions informed its design. All three applications are in operational use at SIXT.

2.3 Research Objectives and Phases

The study has two parts. The first examines the starting point for AI-assisted citizen development at SIXT HR. It asks how far two employees without professional programming experience could take an application, which difficulties they encountered and where they needed support. The second part uses these findings to develop a framework and then evaluates it. The main objective is to assess whether the framework addresses the difficulties that motivated its design. Three objectives, one per phase, guide the study.

1. Establish the starting point and identify difficulties (Phase 1). The first application, the organisation explorer (App 1), shows how development went before the team used the framework. The analysis examines which HR functions the team delivered, what users could do with the application and where development became difficult. User feedback, discarded work, the failure to restore an earlier working version and requests for IT help show the limits of the team's initial approach. These observations define the practical problems the framework has to address.

2. Develop a framework that responds to these difficulties (Phase 2). The difficulties from App 1 are examined together with the literature and formative discussions with citizen developers, users and software engineers. These discussions showed which difficulties other practitioners recognised and which kinds of support seemed feasible at SIXT. The second application, the reorganisation tool (App 2), was used to try selected practices, such as building a prototype before implementation and checking for existing libraries. The resulting framework combines working practices with the organisational and

technical conditions needed to apply them. Its design rationale links each practice or safeguard to the problem it is meant to address (Section 5).

3. Evaluate the framework and identify improvements (Phase 3). The third application, the HR job-profile tool (App 3), is the main case for evaluating the framework as a whole. The evaluation examines which practices the development team applied, how they influenced development and release decisions, and which difficulties remained. It also shows where technical expertise or organisational approval was needed and how SIXT could provide it with reasonable effort. Development records, tests, user feedback and IT reviews show what happened in practice. Final practitioner interviews assess whether the framework addresses problems they recognise, whether its requirements are feasible and what should be changed. The thesis separates changes that were made and tried during development from ideas that came up later but were not tested. The latter are presented as suggestions for future work.

Table 2 summarises how the framework was used across the three applications.

Table 2. Framework use across the three applications.

	App 1 Organisation explorer *Phase 1*	**App 2** Reorganisation tool *Phase 2*	**App 3** HR job profiles *Phase 3*
Overview			
Development period (2026)	June–August	September–mid-October	Mid–end of October
Framework used	None	Selected practices	Final version[a]
Enabling environment			
AI tools	Claude Code	Claude Code, SIXT chatbot	Approved tools in sandbox
Corporate safeguards applied	No	No	Several[b]
Training and Git	No	Partly	Yes
Working practices			
Prototype before building	No	Yes	Yes
Check for existing libraries	No	Yes	Yes
Specification and verification	Unclear	Partly	Yes[c]
Stage gates and roles	No	No	Yes[c]
Traceability (`plan.md`)	No	No	Yes
Cost			
Approx. Claude Code spend	EUR 1,100	EUR 600	EUR 300

[a] Some elements were not applied; see Section 6. [b] See Section 5. [c] Applied, but not documented gate by gate or practice by practice. Spend is shown for context only; it also reflects learning and differences in scope.

The three applications differ in their business functions but are broadly similar in engineering complexity. All use role-based access, handle HR data of similar sensitivity and required only limited integration, including a Workday API connection using OAuth. Complexity was not formally measured.

2.4 Evaluation Criteria

Five criteria follow from these objectives (Table 3). Each criterion is answered with a guiding question and a small set of indicators that can be counted or checked for each application.

Table 3. Evaluation criteria, indicators and sources (own representation).

Criterion	Guiding question	Indicators	Source
Functional fit	Did the application meet the HR need?	• Planned core functions delivered (x of y) • Functions dropped or rejected after user feedback • In operational use (yes/no); number of users	Prototype specification, user feedback, usage data
Development effort	How much effort did development require?	• Elapsed development time (weeks) • Claude Code spend (EUR) • Number of rebuilds or restarts • Features discarded or rebuilt	Calendar, billing data, Git history, team accounts
Reliability and maintainability	How thoroughly was quality checked, and what was found?	• Number of automated tests; share passing • IT review findings, grouped as critical, major or minor • Earlier working version restorable (yes/no)	Test reports, IT reviews, Git history
Accountability and support	Was it clear who checked and approved what, and how much expert help was needed?	• Key decisions recorded with a responsible person (x of y) • Gates held with a named decision-maker (x of y) • Number and topics of requests for IT help	`plan.md`, gate records, IT tickets
Practitioner assessment	How do practitioners judge the framework?	• Participants who consider it clear, useful and feasible (x of 8) • Gaps named by at least two participants	Final interviews

The indicators describe the three applications; they are not statistical measures. With three applications and eight interviews, the numbers show tendencies, not proof. Where an indicator was not recorded, particularly for App 1, it is reported as not available. The grouping of IT review findings into critical, major and minor was done by the researcher, because the reviews used no common rating scale.

Together, these criteria show whether the difficulties identified in App 1 were reduced in App 3, which difficulties remained and what the framework demands from citizen developers and the organisation.

2.5 Application Evidence

The three applications play different roles in the study (Table 1). App 1 establishes the starting point, App 2 shows the use of selected practices while the framework was taking shape, and App 3 is the main case for the final framework. For App 3, the analysis returns to the difficulties identified in App 1 and examines what happened when the corresponding practice was applied. For example, the failure to restore an earlier version of App 1 leads to a specific question for App 3: what evidence shows that version control supported recovery or review? The evidence comes from six sources.

- **IT reviews.** For each application, the development team opened an IT ticket and gave software engineers access to the code. A discussion of about 30 minutes followed, in which the engineers explained strengths, defects and possible improvements. Additional discussions addressed specific needs, such as the Workday integration and the improvements needed for production use. The reviews were qualitative, without a common rating scale. This means that the engineers gave their feedback in their own words; they did not score the applications on a fixed scale. Conclusions are drawn only about what a review actually examined. For example, a review of the code structure says nothing about security unless security was also checked. Accordingly explicit questions were asked regarding access control, privacy, security or operational readiness to acquire evidence that this aspect was actually examined.
- **Development records and tests.** The Git history (App 2 partly, App 3), the tests created and run during development and, for App 3, the `plan.md` file, which records tasks, decisions and open

questions. The value of `plan.md` for governance depends on what a reviewer can trace from it, such as a requirement, a decision or an open check. Responsibility for accepting a risk or approving a release is assessed only where the records name it.

- **Time and cost.** Development periods and approximate Claude Code spend in euros, taken from cloude setting in the resaercher's account. The periods are calendar time, not working hours: the researcher worked part-time as a working student, typically about 20 hours per week, and exact hours per application were not recorded.
- **User feedback.** was collected through walkthrough meetings for App 1. Similar meetings were held for Apps 2 and 3, supplemented by in-app feedback from a broader group of approximately 50 users and feature usage tracking. Users provided quantitative star ratings and qualitative feedback on specific problems. Usage tracking showed which functions people used, while feedback helped identify where they struggled. Together, these sources provided indications of whether users found it easier to complete tasks using the applications. These findings are treated as preliminary indications of business value creation, although stronger claims require concrete evidence of benefits.
- **Requests for help.** IT tickets and other requests for technical support, showing where the team could not proceed alone.
- **The development team's own accounts** of what worked and what did not.

App 1 was built before the framework existed and without version control. Its evidence is therefore partly reconstructed after the fact, from the IT review, user feedback, cost data and the team's recollection. Records made during development are treated as stronger evidence than later recollection, and missing evidence is reported as missing.

2.6 Practitioner Interviews

Practitioner input served two purposes at different points in the study. During Phases 1 and 2, informal formative discussions with two citizen developers and two software engineers helped the researcher understand recurring difficulties and refine the emerging framework. These discussions were not documented in notes; they are used to explain design choices, not as evaluation evidence. In Phase 3, final semi-structured interviews provide the evidence for the criterion practitioner assessment (Table 3) and contribute to SQ3 and SQ4. Where the same people took part in both, their earlier suggestions are treated as design input and their later judgements as feedback on the version presented.

Eight experienced practitioners were interviewed. They were three software engineers (E1–E3) and five citizen developers (C1–C5). They were selected because of their relevant experience (purposive sampling). The citizen developers had each built at least two applications and were familiar with Claude Code. None of the participants belonged to the development team. The software engineers had also carried out the IT reviews, so their views are informed but not independent of them. Following the idea of information power (Malterud et al., 2016), a small sample is sufficient when the aim is narrow and the participants are highly experienced. The study does not claim that the sample is representative.

All interviews followed the same semi-structured procedure. Each interview began with a short slide presentation of the final framework version, followed by an example walkthrough showing how the framework would be used in daily work. Participants were then asked about its clarity, usefulness, feasibility and gaps, and about the resources and organisational conditions it would need. Interviews with software engineers took about one hour and also covered the code quality of the three applications, in order to compare it across the applications. Interviews with citizen developers took about 30 to 45 minutes and focused on whether the framework addresses problems they had experienced. The interview guide is in Appendix [X].

Several measures were taken to reduce bias. Because the researcher designed the framework and also conducted the interviews, participants might have avoided criticism. Every interview therefore asked directly about weaknesses and missing elements. Some participants had given ideas or tried parts of the framework earlier; this is treated as background, not as an additional test, and is kept separate from their final assessment where possible.

The interviews were recorded and handled in line with data protection requirements. Participation was voluntary, and participants agreed verbally to recording and transcription at the start of each interview. Interviews were transcribed with the transcription feature of Microsoft Teams. Generative AI helped summarise the transcripts. The researcher checked every summary against the original statements to avoid inaccurate summaries or wrong conclusions. Only tools approved by SIXT were used. Participants are identified only by their codes (E1–E3 and C1–C5), and the thesis contains no personal HR data.

2.7 Data Analysis and Evidence Integration

The analysis follows the three phases and is organised around the difficulties identified in App 1. For each difficulty, it traces the initial observation, the reason for the framework response, the evidence from applying that response and any remaining limitation.

Phase 1. The App 1 evidence is organised into a list of difficulties. Each difficulty is supported by at least one source, such as user feedback, the IT review, discarded work or a request for help.

Phase 2. Each difficulty is compared with the literature to find practices that address it. The design rationale in Section 5 records, for each practice, the difficulty it addresses and its sources. Results from trying selected practices in App 2 are reported where they changed the design.

Phase 3. The App 3 evidence is analysed in two ways. First, a list shows for each practice whether it was applied, partly applied, not applied or not tested. Second, the App 3 results are compared with App 1 for each evaluation criterion, with App 2 as an intermediate point. The final interviews are analysed with thematic analysis (Braun & Clarke, 2006). Coding starts with the topics clarity, usefulness, feasibility, gaps and organisational conditions, and new themes are added when statements do not fit. The researcher coded all data alone.

Combining the evidence. Application evidence and interview evidence are first analysed separately and then compared. This allows the findings to distinguish three cases: a practice that was adopted, a practice that demonstrably helped in a particular case, and a practice whose value practitioners expected but whose effect remains untested. Different sources can complement each other. For example, a user walkthrough can show that a user completed an HR task, a test with different access roles can show that an access rule held under the tested conditions, and an engineer can explain why this check matters for release. Reported together, these findings connect business function, technical verification and expert judgement, while each keeps its own scope.

Where the sources agree, a cautious conclusion is drawn. Where they disagree, the conflict is retained and reported as a limitation or open question. Where the same engineer reviewed the code and later discussed it in an interview, the interview adds interpretation to that review rather than independent confirmation. Three rules apply: interview opinions are not counted as application tests, expected benefits are not treated as observed results, and a positive IT review does not confirm every governance practice. Each conclusion in Section 6 states whether it rests on application evidence, interview evidence, both, or neither. Suggestions made after the final framework version was fixed on the start of November are reported as future work.

2.8 Research Quality and Limitations

Table 4 summarises the main limitations, their effect and how the study deals with them.

Table 4. Main limitations of the study (own representation).

Limitation	Effect	How it is handled
The researcher designed, applied and evaluated the framework	Interpretation may be biased; other views may be missed	Same questions in all interviews; criticism reported; evidence traceable
The same team built all three applications	Improvements may come from learning, not only from the framework	No causal claims; other factors discussed
App 1 evidence is partly reconstructed	The starting point may be described less precisely	Records preferred over recollection; gaps reported
Formative discussions were informal	Their influence on the design is hard to trace	Used to explain design choices, not as evaluation evidence
App 3 was built in the final weeks	Later stages (testing, pilot, operation) were not observed	Reported as untested
Interviewees saw a presentation, not their own full use	Results show perceived, not observed, usefulness	Kept apart from application evidence
One department in one company	Results may not transfer	Conditions for transfer discussed (SQ4)
No security or data protection specialists available	Specialist assurance is missing	Scope of IT reviews stated; no claim of full assurance
Researcher studies Business Information Systems	The team may be more technical than typical HR staff	Background stated (Section 1.2)

Production readiness can be assessed only within the coverage of the available checks. Where a safeguard, legal requirement, release decision or operational responsibility lacked a documented assessment, its status remains undetermined. Operational use shows that an application entered use; how far it is ready for production depends on which technical and organisational requirements were actually verified.

Taken together, the study offers an exploratory assessment of the framework in one real setting: which difficulties it addresses, what it demands and how practitioners judge it. The cases show what this team achieved under the conditions at SIXT; they provide limited evidence about what another team could achieve independently. The study cannot prove that the framework causes better outcomes. Stronger conclusions require other teams to apply the framework, several independent reviewers and further organisations.

3 Problem and framwork requirments

Insights from building the first software applications and the faced challegnes. Also based on litautre and interviews with citzen develpers. Reverse engineer what the framework solves

4 Theoretical Background and Related Research

4.1 Conceptual Foundations and Key Definitions

AI-assisted, agentic, and vibe coding. AI-supported software development today is not one single thing. It runs along a spectrum defined by degrees of AI autonomy, interaction modalities, and human oversight (Farrag, 2026; Mandl & Mandl, 2026; Sapkota et al., 2025). Grounded in the AI-Augmented Methodology Taxonomy (AAMT) proposed by Farrag (2026), this spectrum can be categorized into three distinct tiers:

1. **Tier 1: Passive Suggestion (AI-Assisted Coding):** At this baseline, Generative AI operates as an inline autocompletion engine within the developer's Integrated Development Environment

(IDE) (Díaz et al., 2026; Farrag, 2026). The AI assists with localized tasks such as filling short code blocks, suggesting syntax, or drafting basic unit tests. The human developer remains the primary code author, maintaining direct cognitive engagement and line-by-line inspection (Díaz et al., 2026).

2. **Tier 2: Active Generation (Vibe Coding):** Coined initially by Karpathy and formalized by Meske et al. (2025), *vibe coding* shifts software creation from deterministic syntax authoring to probabilistic *intent mediation*. Through multi-turn natural language dialogue, developers articulate high-level goals ("the vibe"), while the AI generates entire components or scripts (Farrag, 2026; Meske et al., 2025). In this mode, the primary human role shifts from direct implementation to conversational orchestration and output review (Díaz et al., 2026; Sapkota et al., 2025).

3. **Tier 3: Autonomous Agency (Agentic Coding):** Positioned at the autonomous end of the spectrum, Agentic Software Engineering (ASE) delegates goal-level technical objectives (e.g., refactoring a legacy service or implementing an end-to-end API) to autonomous multi-agent systems (Díaz et al., 2026; Sapkota et al., 2025). Operating within sandboxed execution environments, agentic systems independently formulate execution plans, modify repository files, invoke terminal commands, run test suites, and repair failing code with minimal step-by-step human intervention (Mandl & Mandl, 2026; Sapkota et al., 2025).

The labels used for AI-supported software development are not yet stable. Beyond "vibe coding" and "agentic coding", the literature uses several partly overlapping terms for systems that plan and carry out software tasks with limited human intervention. These include "Agentic Software Engineering" or "SE 3.0" (Hassan et al., 2025), "AI teammates" (Hassan et al., 2024; H. Li et al., 2025), "autonomous coding agents" (H. Li et al., 2025), and "AI software engineer" or "agentic software engineer" (Hoda, 2025). These terms differ in emphasis rather than in the underlying phenomenon. Some describe a tool (*coding agent*), some a role (*AI teammate*, *AI software engineer*), and some a discipline or era (*Agentic Software Engineering*, *SE 3.0*). Hassan et al. (2025), for instance, distinguish one-to-one "agentic coding" from team-level "agentic software engineering". Hoda (2025) observe that terminology drift is already visible even though the field is nascent. Even the term "vibe coding" is defined inconsistently. Sapkota et al. (2025) frame it as a human-in-the-loop counterpart to agentic coding, whereas Ge et al. (2025) define it as the practice of managing coding agents. Because this vocabulary is still evolving and partly shaped by vendor marketing, this thesis does not adopt any single label as authoritative. Instead, it uses the terms defined in Section X as working definitions and characterises each use case by its degree of AI autonomy and human engagement with the code, not by its label.

On one end sits AI-assisted coding, where AI works as a small helper inside the developer's editor (IDE). It suggests the next line of code, fills in short blocks, writes basic tests, or drafts documentation. The developer still writes and reads the code. On the other end sits agentic coding. Here, the developer gives the AI a high-level goal—for example, "migrate this repository from Python 2 to Python 3"—and the AI plans the steps, opens files, runs tests in a safe sandbox, reads the error logs, and fixes itself with little human help (Galster et al., 2026; Sapkota et al., 2025). Between and around these two sits vibe coding (VC), a term coined by Andrej Karpathy in February 2025 (Karpathy, 2025). Sarkar and Drosos (2025) describe vibe coding as "material disengagement": the programmer no longer works directly on the code itself but steers the AI through natural-language chat, screenshots, and quick "vibe checks" on whether the running app feels right.

From chat window to terminal: YOLO, Claude Code, Codex, and MCP. For a long time, using an LLM for coding meant copy-and-paste: the developer copied code into a chat window, got a suggestion, and pasted it back. Newer tools remove this manual loop. Claude Code and OpenAI Codex CLI run directly in the terminal. They can read the whole repository, edit files, run shell commands, and execute tests on their own (Galster et al., 2026). This is often used in "YOLO mode" — the developer accepts all changes without reading each diff, trusting the agent to move forward. On top of this, the Model Context Protocol (MCP) lets the agent connect to external tools and data sources (databases, ticket systems, documentation, other MCP servers) through a standard interface (Galster et al., 2026). In parallel, sub-agents can be spawned for specific jobs (e.g., a reviewer agent, a test-writer agent) and only their final output flows back to the main agent (Galster et al., 2026). This is a real historical shift: from "suggest a snippet in a chat window" to "work autonomously inside my codebase with access to my tools".

Deterministic tools vs. probabilistic foundation models. To understand why this shift matters, it helps to look at what came before. In classical software engineering — and in low-code / no-code (LCNC) plat-

forms — the tools are deterministic. This means the same input always gives the same output, following fixed, hard-coded rules. LCNC platforms use Model-Driven Development (MDD): a citizen developer drags visual blocks onto a canvas to draw the business logic, and the platform translates this visual model into runnable code or database configurations through strict compiler-like rules (François et al., 2025; Gadde, 2025). There is no guesswork. The compiler acts as a strict guardrail: it does not let the user produce an invalid SQL statement or an unclosed bracket.

Generative AI works differently. Large Language Models (LLMs) are probabilistic. They do not ”know” syntax the way a compiler does. Instead, they use autoregressive next-token prediction: for every next word (token) in the output, they compute the statistical probability of what comes next, based on patterns learned from very large training corpora of code and text (Mandl & Mandl, 2026). Two consequences follow. First, the same natural-language prompt can produce different code on different runs — the model is non-deterministic by design. Second, because the output is what is statistically plausible and not what is mathematically verified, LLMs often produce code that looks clean and professional but contains hallucinated libraries, security holes, or architectural inconsistencies (Farrag, 2026).

Why this matters for the SIXT thesis: the Productivity–Reliability Paradox. This deterministic vs. probabilistic split is the root cause of the Productivity–Reliability Paradox (PRP) described by Farrag (2026). Low-code has built-in guardrails and is therefore safe but slow and narrow. Vibe coding is fast and open but has no built-in guardrails; if the developer blindly accepts every suggestion, the codebase quickly fills up with hidden or ”shadow” technical debt (Farrag, 2026; Kobiella et al., 2026). The idea of this thesis is to bridge the two: to take a very fast but volatile probabilistic engine (vibe coding) and wrap it inside a strict deterministic harness — CI/CD pipelines, automated security tests, and expert stage-gates — so that all AI-generated code must pass through fixed, machine-checked rules before it reaches production. In the terms of Galster et al. (2026), this is harness engineering: configuring the environment around the model (context files, skills, sub-agents, MCP tools, hooks) so the model can move fast without breaking the rules of the enterprise. The goal is simple: the speed of vibe coding, with the safety of classic software engineering.

4.2 Software development before and after GenAI REVIEWED

Developing a governance and development framework for citizen developers requires an understanding of how generative artificial intelligence (GenAI) changes the software development life cycle (SDLC). This includes both the activities needed to turn a business need into operational software and the distribution of responsibilities across product, design, engineering and operations.

Before the widespread adoption of GenAI coding tools, a representative iterative development cycle began with identifying business and user needs and translating them into prioritised requirements. Product managers and product owners coordinated priorities, designers developed interaction concepts, and software engineers implemented the solution. Implementation involved repeated testing and debugging, followed by code review, integration and deployment. Once the software entered production, operational monitoring, incidents and user feedback informed corrections and new development tasks. These activities formed a continuous feedback cycle rather than a strictly linear sequence. The precise division of responsibilities varied between organisations. This description provides a simplified baseline for the lifecycle activities discussed by Mandl and Mandl (2026) and the role-specific handoffs examined by J. Li et al. (2026). Figure 1 summarises this baseline software factory.

Code implementation required substantial time and technical expertise, but it should not be described as the sole universal bottleneck of traditional software development. Delivery also depended on requirements clarification, review, integration, testing and release. Accelerating implementation alone therefore cannot guarantee an equivalent improvement in delivered software. Demirer et al. (2026) explain this through the weak-link hypothesis: gains in one activity are constrained by complementary activities elsewhere in the production process.

With GenAI, users can express requirements in natural language and ask coding assistants or agents to generate, modify and test implementations. Meske et al. (2025) describe this as a shift in how human intent is translated into software: people increasingly communicate their goals through dialogue, while AI systems infer and produce the corresponding implementation. In product teams, J. Li et al. (2026) identify an iterative workflow comprising ideation, generation, debugging, and testing and review. These activities can overlap more closely because a functional prototype can be generated and evaluated directly,

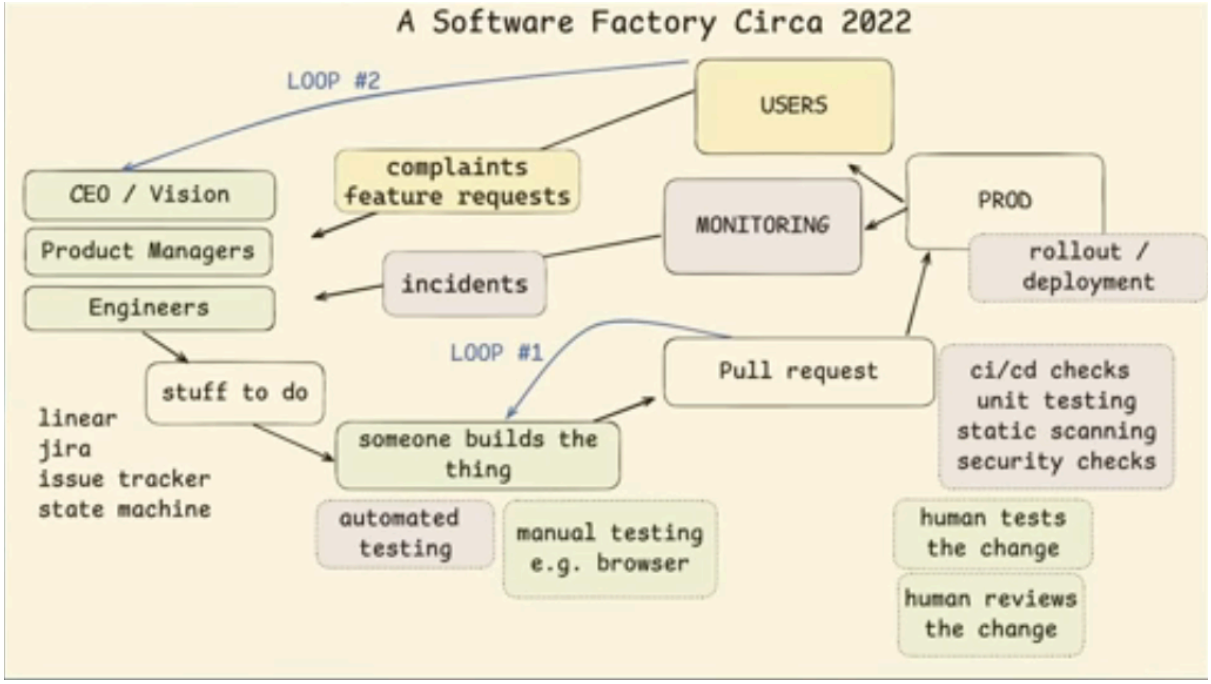


Figure 1. A software factory circa 2022: human roles plan work, engineers build and test, and changes reach production via pull requests, CI/CD checks, and human review. Feedback returns through monitoring, incidents, and user complaints or feature requests (BCG Platinion, 2026).

reducing some of the handoffs between initial ideas, design and implementation. Figure 2 illustrates the agentic software factory in which agents perform much of the build and testing work.

The wider development lifecycle nevertheless remains necessary. Generated changes must still be assessed against requirements, checked for technical quality, integrated with existing systems and prepared for deployment. After deployment, monitoring and user feedback continue to identify defects and new requirements. GenAI can support these activities, including test generation, review, diagnosis and maintenance, but its outputs still require appropriate verification and accountable decisions (Mandl & Mandl, 2026). Faster prototyping consequently does not establish production readiness: Kobiella et al. (2026) find that non-technical users can create useful prototypes while continuing to encounter substantial barriers in deployment and long-term maintenance.

As code generation accelerates, review, verification and integration can become more prominent bottlenecks. In their working paper, Demirer et al. (2026) report cumulative increases of approximately 240% in commits across successive generations of AI coding tools, compared with approximately 30

A related challenge concerns compounding error and compounding correctness, illustrated in Figure 4. These terms are used here as a conceptual synthesis rather than as established quantitative laws. Compounding error describes a pattern in which later changes build on unresolved defects or incorrect assumptions, increasing the difficulty of subsequent diagnosis and correction. This interpretation is consistent with the risks identified by Meske et al. (2025), including unpredictable rewrites, unintended side effects and accumulating technical debt. Compounding correctness describes the intended alternative: preserving verified behaviour while introducing controlled changes. For example, S. Wang (2026) propose explicit task-level contracts that connect requirements, generated code and continuous verification. Such mechanisms can support cumulative quality improvement, but passing checks cannot guarantee correctness or maintainability.

GenAI also makes some professional boundaries more permeable, particularly during exploration and prototyping. J. Li et al. (2026) report that designers, engineers and product managers can explore and develop prototypes with fewer sequential handoffs. Product owners and business specialists can therefore participate more directly in implementation, while engineers can use AI assistance to explore interfaces and work across technical tasks. However, this expansion of participation does not make professional expertise interchangeable. Kobiella et al. (2026) show that non-technical users' ability to

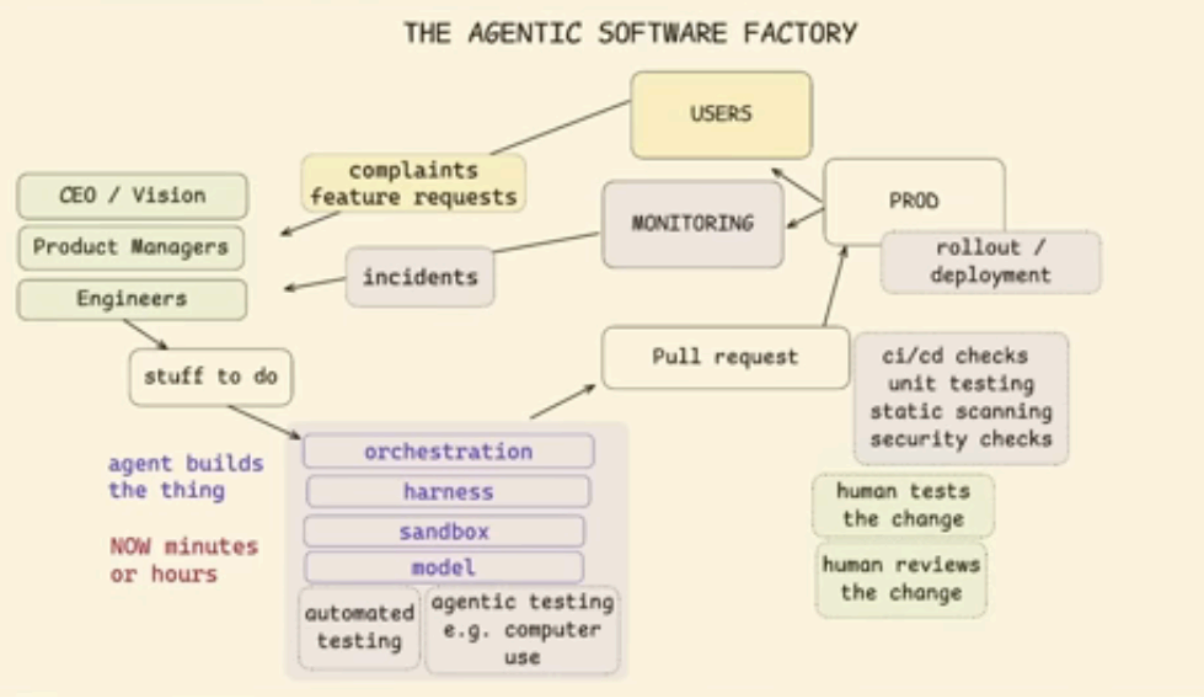


Figure 2. The agentic software factory: agents perform the build and much of the testing (orchestration, harness, sandbox, model), compressing implementation to minutes or hours, while humans retain review and testing before rollout to production (BCG Platinion, 2026).

produce and assess visible prototype behaviour does not extend equally to evaluating underlying code quality, security or maintainability. For this thesis, these findings motivate a framework that combines broader participation with explicit responsibilities and verification requirements. Citizen developers can contribute domain knowledge, explore solutions and assess whether prototypes meet business needs. Progress towards production requires additional technical, organisational and operational support, alongside appropriately qualified review.

Beyond supporting individual development tasks, a longer-term ambition is to automate much of the software lifecycle through coordinated AI agents. In this vision of an agentic software factory, agents translate approved requirements into implementations, execute tests, address failures, and support deployment and maintenance within defined constraints. The prospect of connecting these activities into an increasingly autonomous process helps explain the growing interest in agentic software factories (Mandl & Mandl, 2026). However, extensive automation remains an aspiration rather than an established capability across enterprise contexts. Human goal-setting, governance and accountability remain necessary, even where execution becomes largely autonomous (Mandl & Mandl, 2026; Meske et al., 2025). For this thesis, this vision provides a direction of development; the framework must establish which activities can already be delegated reliably and where expert intervention remains required.

4.3 Capabilities and Limitations of Non-Technical Developers

4.4 Expert Support in Citizen-Led Software Development

4.5 Governance, Guardrails, and Development Support

4.6 Conceptualisation of Production Readiness

4.7 Synthesis of the Literature and Research Gap

5 The Governance and Development Framework

5.1 The Enabling Environment

5.1.1 Technology Platform

5.1.1.1 Security rules and checks

IT defines the security rules for all citizen-developed applications once, and they apply to every change. Following the harness engineering view summarised by **TODO-koul2026vibesec<empty citation>**, each rule exists in two forms. As a *guide*, it is written in the system specification, which every agent reads before it acts, so that secure code becomes more likely from the start. As a *sensor*, it is enforced by a deterministic check in the automated pipeline, which examines the result and blocks the change if the rule is broken. Only the sensor makes a rule binding, because agents can ignore or override instructions, especially when a user pushes back (**TODO-koul2026vibesec**). In the Thoughtworks case behind this source, an AI recommended making a storage bucket public and gave a service account far broader rights than the task required; in both cases, a person asking the right question stopped it (**TODO-koul2026vibesec**). Citizen developers cannot be expected to ask these questions reliably, which is why the framework moves them into rules and checks. Table 5 lists the rule areas. They follow the security context file described by **TODO-koul2026vibesec<empty citation>**, extended by an allow-list for libraries (Section 5.2.3.3.5) and the OWASP Top 10 as a basis for code analysis (**TODO-owasptop10**). The concrete rules and tools differ between organisations; the table shows the categories, not a complete policy.

Table 5. Security rules as guides in the system specification and as sensors in the pipeline (own representation, rule areas based on TODO-koul2026vibesec<empty citation>).

Area	Guide: rule in the system specification	Sensor: check in the pipeline	Checked at
Least privilege	Every account, role and storage receives only the rights its task needs; no public storage	Validation of the configuration blocks public storage and broad roles	Each change; G4
Passwords and keys	No passwords, keys or tokens in the code; only the approved password manager	Secret scanning	Each task (quick); each change (full)
External libraries	Only libraries on the list approved by IT; requests for new ones go to IT	Allow-list check and dependency scan	Each change; operation
Secure code	Avoid the weaknesses listed in the OWASP Top 10, for example unchecked user input	Static analysis with rules set by IT	Each change; G4
Access rights	Every function checks the user's role	Role tests from the role matrix	Each change; G4
Accountability	No AI-generated change is merged without independent tests and technical approval	Branch protection: merging requires passed checks and an approval	Checkpoint B

The rules are versioned with the system specification and changed only by IT. When a security check or

a reviewer finds the same problem repeatedly, IT turns it into a new rule, as described for the Evolve step of SCOPE-V.

5.1.2 Governance Structure

5.2 Working Practices

5.2.1 Core Concepts and Principles

5.2.2 The Stage-Gate Process Foundation REVIEWED

AI-assisted development changes where the cost of software lies. Generating code has become fast and cheap (see paragraph software dev before and after GenAI TBD). Specifying, verifying, integrating and taking responsibility for that code has not, and these activities now make up a larger share of the total cost of a change (Z. Wang & Liu, 2026). When more code is produced than can be checked, teams build up *verification debt*: weak tests, hidden regressions and a growing review burden. The time needed for review and clean-up can cancel out the speed gained through AI (Koch, 2026).

For citizen developers, these risks are higher. They can judge whether software looks and behaves as intended, but not whether the code is secure or maintainable. They also find the step from a prototype to a deployed product especially difficult (Kobiella et al., 2026). Applications built outside IT can turn into shadow IT and technical debt that the organisation later has to maintain (Viljoen et al., 2024).

These conditions call for an approach that limits investment until uncertainty has been reduced. A stage-gate process does this. Work is divided into stages, and a gate between two stages decides whether the work may continue. Each gate unlocks more resources: time, budget, data access, technical support and users. Early stages are cheap and tightly restricted. Later stages receive more support but must show more evidence. An idea therefore does not receive production-level support just because it is attractive, and a prototype is not released just because it works.

Gates also make it possible to match oversight to risk. Human oversight adds value, but it is costly when applied to every task in the same way (Kang, 2026). Reviewing everything in depth would overload scarce experts, while reviewing too little would let risky applications reach users. In addition, each gate has a named decision-maker. This prevents an application from being released before anyone has accepted responsibility for its remaining risks (Z. Wang & Liu, 2026).

The process must, however, remain lightweight. Excessive control can demotivate citizen developers, whose participation is often voluntary (Viljoen et al., 2024). The framework therefore keeps the number of stages small, uses the same decision logic at every gate and lets low-risk applications pass with little effort.

The logic of deciding early before investing more is also found in MeRGE, the Method for Responsible GenAI-Based Information Systems Engineering. Its first phase checks whether a use case is realistic, in order to avoid unnecessary, time-consuming and resource-intensive work later (Fetzer et al., 2026). MeRGE was, however, not designed for vibe coding or citizen development. It guides organisations in engineering information systems that contain GenAI, such as an HR chatbot based on a large language model. Fetzer et al. (2026) developed it in a twelve-month action design research project at a German premium car manufacturer. This thesis addresses a different object: not software that contains GenAI, but software that non-technical employees build using GenAI. MeRGE therefore does not cover the controls that matter most here, such as how AI-generated code is verified against requirements or how a citizen-built prototype moves into production development. It is also not a formal gate framework with pass and fail criteria, and it excludes the maintenance of systems after deployment (Fetzer et al., 2026).

MeRGE is nevertheless used in this thesis because its organisational logic does not depend on this difference. Deciding early whether a use case is worth pursuing, linking business value with data sensitivity and risk, involving experts at defined points and introducing software through a controlled pilot apply to any internally developed software. MeRGE is also close to the setting of this thesis: it was developed with action design research in a large German company and includes an HR use case. This thesis therefore adopts MeRGE selectively. Its Precheck and Conceptualization phases inform the early screening at G1 and G2, and its Quality and Deployment phases inform the readiness checks, approvals and pilot

at G4 and G5. Its GenAI-specific measures, such as protection against harmful outputs, are not adopted. The transition from prototype to production, the controls for AI-assisted coding and the operation stage are added by this thesis.

5.2.2.1 Overall structure

The framework moves an application through six stages, each followed by a gate (Table 6).

Table 6. Stages and gates of the framework (own representation).

Stage	Main activity	Gate question
1 Screen and scope	Check suitability, value, data and risk; set limits for the experiment	G1: May the idea be explored?
2 Prototype	Clarify the need with a prototype and validate it with users	G2: Is production development justified?
3 Build and verify	Build the application in small changes, each verified in an Agile-V cycle	G3: Is the planned scope complete and accepted?
4 Test application	Test the whole application: end-to-end, security, data protection, user acceptance and operational readiness	G4: Is it ready for real users?
5 Pilot	Use with a limited group; measure value and support effort	G5: Should it be released more widely?
6 Operate and reassess	Monitor, maintain and change the application; reassess value and risk	G6: Continue, change, hand over or retire?

Stages 2 and 3 are described in detail in Sections TBD. The other stages build on MeRGE, which moves from an early check through conceptualisation, development and quality assurance to deployment (Fetzer et al., 2026). The framework departs from MeRGE in two ways. First, it adds a separate test stage for the whole application, because accepting individual changes does not show that the application works as a whole. Second, it adds an operation stage, because MeRGE excludes maintenance (Fetzer et al., 2026), while AI-built applications in particular risk losing the shared understanding of why they exist and how they work (Storey, 2026).

The stage-gate process serves as the macro framework of this thesis. It steers the application as a whole and decides at each gate whether it may advance. Within individual stages, it adopts smaller frameworks that guide the detailed work. In stage 2, a lightweight V-shaped cycle adapted from Agile V guides how the prototype is specified, built and evaluated (Koch & Wellbrock, 2026). In stage 3, a fuller Agile-V cycle guides how each change is specified, built and accepted, and within each change the SCOPE-V cycle guides how an agent completes a single task (Koch, 2026; Koch & Wellbrock, 2026). These smaller frameworks decide only about their own level of work. Accepting a change therefore never authorises a release; only the gates of the macro framework do.

5.2.2.2 Gate decisions

Every gate uses the same five outcomes:

- **Go:** the work moves to the next stage and receives the resources it needs.
- **Revise:** the work stays in the current stage until open points are resolved.
- **Restrict:** the work continues within narrower limits, for example fewer users or no personal data.
- **Hand over:** the work moves to IT or professional developers, because it exceeds what citizen development can safely deliver.
- **Stop:** the work ends, and what was learned is recorded.

Decisions are based on evidence, not on impressions. This follows the acceptance rule of Agentic Agile-V: AI-generated output is accepted not because it looks plausible but because it provides the evidence

required for its risk level (Koch, 2026). Each stage therefore has *entry criteria*, which the previous gate must have confirmed, and *exit criteria*, which the next gate checks. Stop criteria, such as an exceeded budget, are defined at the start. Although the stages are shown in sequence, work may return to an earlier stage when new findings require it, as MeRGE also recommends (Fetzer et al., 2026).

5.2.2.3 Risk-based decisions

How strict a gate is depends on risk. The framework assigns each application a risk tier from R0 (exploratory) to R3 (high assurance), based on the levels proposed in Agentic Agile-V (Koch, 2026). The tier is set at G1, confirmed at G2 and reviewed at later gates. It decides who must approve and how much evidence is required. If the tier is uncertain, the higher tier applies, following the fail-safe principle of Kang (2026). Applications in the highest tier are not developed by citizen developers alone but handed over to IT or developed jointly. Individual changes in stage 3 receive their own tier, which may be lower than the tier of the application but never higher. The tiers and their criteria are explained in Section TBD.

5.2.2.4 Roles and responsibilities

The framework distinguishes seven roles (Table 7). In the setting of this thesis, the end user, the citizen developer and the business owner are often the same person: an HR employee who builds a tool to solve a problem in their own daily work. These roles can, however, be separated. The end users may be colleagues in the same department or in another team. The citizen developer may not use the application much themselves but be responsible for the problem it solves, or may support a product manager or process owner in building it. The roles are therefore described separately, even though one person can hold several of them. When they overlap, the separation of building and approving still applies to the risk tier and the technical approval, which always remain with IT and the relevant specialists.

Table 7. Roles in the stage-gate framework (own representation).

Role	Responsible for	Does not
End user	Using the application, giving feedback in prototype walkthroughs, user acceptance tests and the pilot	Approve releases
Citizen developer	Business requirements, prototype, feature specification, business acceptance	Set the risk tier or give technical approval
Business owner	Business value, budget and overall accountability for the application	–
IT reviewer	System specification, technical platform, technical approval	Define business requirements
Specialists	Security, data protection and employee-representation checks when triggered	Review routine changes
Operations owner	Support, monitoring and incidents after release	–
AI agents	Planning, building, testing and compiling evidence	Approve anything

Four principles guide this distribution. First, business ownership stays with the business: the citizen developer defines what the application should do, and the business owner remains accountable for it. Second, technical and legal judgement stays with specialists. This mirrors the division in spec-driven development, where technical leads own the system specification and developers own the specification of their change (Díaz et al., 2026). Third, building and approving are separated: whoever produces a result does not approve its technical quality or its release (Kang, 2026). The citizen developer may therefore accept a change from a business perspective, but does not set its risk tier or approve it technically. AI agents never hold approval authority, also because their claims that a task is complete are not always reliable (Al Hasan & Biswas, 2026). Fourth, every gate has a named decision-maker, and the set of decision-makers grows with the risk tier.

The stages, gates, roles and tiers described here combine ideas from the cited sources, but their specific arrangement is a design choice of this thesis. Whether it works in practice, and whether its effort is

acceptable for citizen developers and experts, is evaluated through ADR (Section TBD).

5.2.3 The Stages of the Stage-Gate Process REVIEWED

5.2.3.1 Screen and scope

The first stage decides, with little effort, whether an idea should be pursued through citizen development and under which limits. It produces a short description of the idea, a provisional risk tier and the boundaries for the prototype experiment in stage 2. The citizen developer and the business owner describe the idea. The IT reviewer assesses its technical fit and sets the risk tier. Specialists are involved only when the screening raises legal or data protection questions.

5.2.3.1.1 Problem and business case

The citizen developer first describes the problem, the intended users and what success would look like, before discussing any solution. MeRGE uses a use-case canvas for this purpose, covering the problem, the value proposition and the risks (Fetzer et al., 2026). IACDM likewise argues that the problem must be understood before technical decisions are made (Moreira, 2026).

The business owner then compares the expected benefit with the expected effort. AI makes development cheaper, so problems that used to be too small for professional development may now be worth solving. It does not, however, make development free. The effort for specification, review, testing and later maintenance remains and is easily underestimated (Z. Wang & Liu, 2026). In a controlled trial, experienced developers even worked more slowly with AI while believing they were faster (Moreira, 2026). The estimate therefore includes the time of the citizen developer, tool costs, the review effort of IT and specialists, and the operation of the application after release. Expert time, not code generation, is usually the scarcest resource.

5.2.3.1.2 Suitability for citizen development

Not every application should be built by citizen developers. Research on low-code development shows that IT should co-develop or lead when an application requires complex integrations, must meet enterprise standards or demands technical expertise.Mission-critical or cross-functional processes also need more standardisation than isolated tasks (Viljoen et al., 2024). Although this research concerns low-code platforms rather than AI-assisted coding, the same reasons apply. They are even stronger here, because citizen developers cannot inspect the generated code (Kobiella et al., 2026). MeRGE shows a related decision: the planned SAP integration of the HR assistant was postponed because it would have raised the criticality of the processed data (Fetzer et al., 2026). Studies with non-programmers point in the same direction. They performed well on small, fully specified tasks (Esnaola et al., 2026), but features that combined several systems required many iterations (Fortes-Ferreira et al., 2025). Table 8 lists the warning signs used in this stage.

Table 8. Warning signs for citizen development (own representation).

Warning sign	Typical outcome
Integration with or changes to large enterprise systems, such as SAP or Workday	Hand over or co-develop
Write access to systems of record, such as payroll or employee master data	Restrict scope or hand over
Changes to legacy code or existing IT-owned applications	Hand over
Mission-critical or cross-departmental process	Co-develop
Large scope with many user roles, modules or interacting features	Split or restrict

A warning sign does not always end an idea. Often the scope can be reduced so that the risky part is left out or postponed, as in the MeRGE example.

5.2.3.1.3 Reuse and technical complexity

Before anything new is built, the citizen developer and IT check whether a similar application, an existing tool or a standard product or open source software already solves the problem. MeRGE includes this as a make-or-buy decision that considers the technologies already available in the organisation (Fetzer et al., 2026). IACDM applies the same idea to individual components: if a mature library exists, it should be used; if not, a small proof of concept comes first (Moreira, 2026). This matters for AI-assisted development, because AI agents tend to produce plausible new code for problems that already have a tested solution (Moreira, 2026).

IT then judges the technical complexity: the required integrations, the data model, the number of user roles and whether the application fits the approved platform and technology stack. IACDM suggests judging complexity from observable signals rather than from estimated code size, with a simple rule: if a project cannot confidently be called simple, it should be treated as complex (Moreira, 2026).

5.2.3.1.4 Data, risk and legal requirements

The data that the application would process is classified first, as in the first phase of MeRGE (Fetzer et al., 2026). Data is, however, only one factor in the provisional risk tier (Section TBD). Following Kang (2026), who classify AI-generated code by data sensitivity, regulatory impact, proximity to customers and reversibility, the tier also considers who uses the application, whether it only reads from other systems or also writes to them, how serious and how reversible errors would be, and how much a business process depends on it. An error in a personal to-do tool is minor, while an error that affects pay, contracts or decisions about employees is not. Security-sensitive functions, such as authentication or access rights, also raise the tier (Koch, 2026), as do the warning signs described above. The highest factor determines the tier. The tier is set by IT, not by the citizen developer, and if it is uncertain, the higher tier applies (Kang, 2026). Internal policies and legal requirements, such as data protection rules or the involvement of the works council, are identified early, because they can make a use case impossible to realise (Fetzer et al., 2026). The selection of these factors is a design choice of this thesis and is refined through ADR.

5.2.3.1.5 Capability of the citizen developer

Citizen developers differ in their technical knowledge, their ability and willingness to learn, and the time they can invest. These differences shape what they can build safely. Users with basic coding knowledge were able to make small changes to generated code themselves, while non-technical users struggled (Kobiella et al., 2026). Non-programmers also often lacked the vocabulary to express their intent (Feldman & Anderson, 2024). Low-code research therefore recommends tailoring governance to the people who build (Viljoen et al., 2024), which also apply here in this case. The following examples show how specific skills change what a citizen developer can do with a coding agent such as Claude Code:

- **Terminal and coding agent.** Claude Code runs in the terminal and asks for permission before it runs a command or edits a file (Galster et al., 2026). A citizen developer who does not understand these commands cannot judge whether a request is harmless, such as running the tests, or risky, such as deleting files or installing an unknown package. They are then likely to approve every request, which amounts to the "YOLO mode" described in Section TBD. Someone with basic terminal knowledge can recognise risky commands and reject them.
- **Git.** Suppose a new feature breaks a part of the application that worked yesterday. Without Git, the citizen developer must ask the agent to repair its own mistake, which may cause further changes, or start again. With basic Git knowledge, they can return to the last working version in one step, compare what has changed and keep the specification and plan versioned together with the code.
- **IDE.** Agents sometimes change files outside their task or claim that a task is complete when it is not (Al Hasan & Biswas, 2026; Uddin, n.d.). Without an IDE, the citizen developer must rely on the agent's own summary. In an IDE, they can see which files were changed, notice an unexpected change to an unrelated part of the application, run the tests and read the error messages themselves.
- **Basic coding concepts.** A citizen developer who understands functions, data structures or interfaces can follow the technical part of the plan, notice obvious mistakes, such as a password written directly into the code, and give the agent more precise instructions.

- **Text only.** A citizen developer without any of these skills can still describe requirements and judge a prototype in stage 2. For production development, however, they would depend on IT for every technical step.
- **Agents and context windows.** A coding agent only knows what is in its current context: the conversation so far, the files it has read and its rule files. Model performance tends to decline as this context grows longer and mixes unrelated information (Hong et al., 2025), and earlier prompts can keep steering later output even after the requirements have changed (Sarkar & Drosos, 2025). A citizen developer who does not know this may keep working in one very long chat and wonder why the agent forgets earlier decisions or repeats old mistakes. Someone who understands context windows knows when to start a fresh session, why the framework hands work over through the specification and plan.md rather than through the chat history, and why each agent role works in its own session (Section TBD).

At a minimum, every citizen developer must understand the basic concepts of software development (e.g., how Git works, how to code with agents, and what a terminal is), as well as the process behind this framework (e.g., why specifications are written, what the checkpoints and gates decide, and how evidence is used). Without this understanding, the work is likely to drift back into unstructured vibe coding. This thesis therefore proposes a short capability assessment, for example a self-assessment followed by a brief conversation with an IT mentor. Based on the result, the risk tier and the complexity of the application, the citizen developer may be required to complete certain IT courses and pass their tests before the relevant stage begins, for example a course on the framework process before stage 2 and a course on Git and the coding agent before stage 3 (Section TBD). The assessment does not decide who may take part. It decides which training and support are needed, such as courses, mentoring or co-development with IT, and up to which risk tier the citizen developer can work without close support. The tests confirm the knowledge taught in the courses and are not used to evaluate employees' performance. The assessment and the course requirements are design proposals of this thesis.

The capabilities of citizen developers should not be seen as fixed. With GenAI itself, online learning resources and internal training, almost all of the concepts above can be learned by citizen developers who are willing to learn and have the time to do so. A citizen developer can, for example, ask an AI assistant to explain an unfamiliar command before approving it, or to walk them through a Git conflict step by step. Learning can also happen just in time, when a concept is first needed, with support from an IT mentor or an internal assistant. This helps beginners overcome blockers quickly and allows more experienced citizen developers to deepen concepts they understand only partly. The capability assessment therefore describes a starting point rather than a permanent limit, and the support level can be adjusted as the citizen developer gains experience. However, whether this learning effort is economically feasible remains an open question. If a citizen developer needs three months to learn enough to build a simple application, the effort may outweigh the benefit, and handing the work over to IT may be the better choice. For this reason, the expected learning effort is included in the business case at G1. This thesis does not examine in detail how much learning effort is acceptable.

5.2.3.1.6 Infrastructure

Citizen development is only safe when the technical and governance infrastructure described in Section XX is in place. Citizen developers need controlled sandboxes and separate environments. Where a platform lacks such safeguards, organisations must add access controls and review gates (Viljoen et al., 2024). A practitioner in the same study warned that people with little programming experience and unrestricted access to company data could unintentionally expose sensitive data (Viljoen et al., 2024). Stage 1 therefore checks whether the infrastructure required for the risk tier is available. If it is not, the idea is restricted to mock-ups with synthetic data or stopped.

5.2.3.1.7 Experiment boundaries and gate G1

If the idea passes the screening, the business owner and IT agree on the boundaries of the prototype experiment: a time limit, a budget, the approved tools, the data that may be used (normally synthetic or anonymised) and the stop criteria. In line with the exploratory risk level R0, no production credentials are used (Koch, 2026). They also agree whether IT will enable, oversee or co-develop, since this affects later staffing and support (Viljoen et al., 2024).

At G1, the decision-makers check whether:

- the problem, users and success criteria are described;
- no warning sign remains unresolved;
- no existing solution fits better;
- the complexity matches the capability and support of the citizen developer;
- the expected benefit justifies the expected effort;
- the data is classified and a provisional risk tier is set;
- the infrastructure required for this tier is available;
- the experiment boundaries and stop criteria are agreed.

If all criteria are met, the idea moves to stage 2 as an R0 experiment. Otherwise, the gate decides to revise, restrict, hand over or stop. The depth of the screening depends on the risk: for a small R0 idea, a one-page description and a short conversation with IT are sufficient.

5.2.3.2 Prototype

The framework uses a lightweight, V-shaped exploration cycle during prototype development and evaluation. It selectively adapts the connection between requirements and evaluation found in Agile V (Koch & Wellbrock, 2026). It is a design adaptation of this thesis, rather than the complete Agile-V process. The SCOPE-V task cycle, separate verification agents and detailed engineering evidence are reserved for the subsequent production-development stage, as they are not needed at this stage and would only create extra work. The Agile-V framework was chosen because AI-assisted development shifts the main challenge from generating code to verifying that the resulting software meets business requirements, functions reliably and satisfies quality standards. Its main advantage is the direct connection between requirements and verification, ensuring that each development decision can be evaluated against predefined expectations rather than relying on plausible AI-generated outputs. Combining the V-Model's structured verification with Agile's iterative flexibility makes it suitable for both development stages: during prototyping, a lightweight V-cycle supports rapid exploration, feedback and validation of business needs; during production development, more rigorous task-level V-cycles introduce independent testing, traceability and documented evidence. This allows the same underlying principle—verification before acceptance—to guide both stages while adapting the level of control to their different objectives and risks.

Koch (2026) propose converting exploratory dialogues with different LLMs into a reviewed execution brief before starting the develeopment (in this case, the prototype creation), because long conversations can contain superseded assumptions and implicit constraints. This thesis operationalises that principle through a specification handover into a fresh chat or project context before building the consolidated prototype. Similar practices have been observed in vibe coding: developers deliberately opened a new chat or cleared the AI's context before starting a new phase of work (Sarkar & Drosos, 2025), and non-technical participants used a general-purpose chatbot to turn their ideas into a structured prompt or prototype description for a prototyping tool (Kobiella et al., 2026).

The prototype may consist of screens, a clickable mock-up or a limited interactive demonstration created with an AI-assisted design or prototyping tool. Its purpose is to clarify the business need, explore the proposed workflow and gather early user feedback before committing to production engineering.

This step of prototyping is especially critical for non-technical citizen developers, who face distinct communicative and evaluative barriers when building software with AI. In a controlled study with 67 non-programmers, participants struggled to express their intent in natural language and lacked the technical vocabulary that code-generating models respond to (Feldman & Anderson, 2024). In a case study, a person without coding experience could not produce a working driving simulator with a single prompt. Features that combined several systems required many iterations, and vague prompts led to unstable results (Fortes-Ferreira et al., 2025). Even experienced programmers found it difficult to describe visual ideas, such as diagrams, in words (Sarkar & Drosos, 2025). Conversely, non-programmers assisted by a conversational AI performed comparatively well when tasks were small and fully specified with business rules and test cases (Esnaola et al., 2026).

A prototype addresses these barriers. Instead of describing elements such as navigation, filters or button states in words alone, the citizen developer can show, inspect and correct them step by step. Kobiella et al. (2026) found that AI prototyping tools enabled non-technical users to create high-fidelity prototypes for exploration and stakeholder communication. The same study shows why this form of review suits citizen developers. Non-technical participants could judge whether the output looked as intended, but not whether the underlying code was good, performant or secure (Kobiella et al., 2026). Prototyping therefore relies on a form of validation that citizen developers can actually perform.

Furthermore, a clearly defined prototype is likely to reduce changes during production development. Participants in Kobiella et al. (2026) contrasted the relative ease of building and adding features with the difficulty of modifying or removing them. People with basic coding knowledge could make minor adjustments directly in the code, but non-technical users struggled. One participant noted that without coding experience, even small layout changes would require "even more AI calls" (Kobiella et al., 2026). They also gained little from reading generated code or error messages and when their attempts to fix or improve the code using LLMs failed, many ran out of strategies for what to try next (Feldman & Anderson, 2024). Non-technical users also reported that moving from a prototype to a deployed, working product was very challenging (Kobiella et al., 2026). Clarifying what should be built before production development therefore matters more for citizen developers, because later corrections are harder for them to make. This difficulty can be explained through two possible development patterns: compounding error and compounding correctness. These terms, which are common in the practice of AI-assisted development (StrongDM AI, 2026), are used here as a synthesis of sources rather than as established or quantitatively validated laws (see Figure 4).

For the proposed framework, these ideas support placing a prototyping step between the idea and production development especially for citizen developers. During prototyping, the specification and user walkthroughs help identify misunderstandings about what should be built and should reduce rework and improve outcomes for citizen developers. This will be evaluated through ADR.

Since AI-generated code can appear correct while containing hidden logical errors S. Wang (2026), we adopt a V-shaped framework in which requirements are defined on the left, the prototype is built at the bottom, and its behaviour is systematically tested against those requirements on the right (see Figure 3).

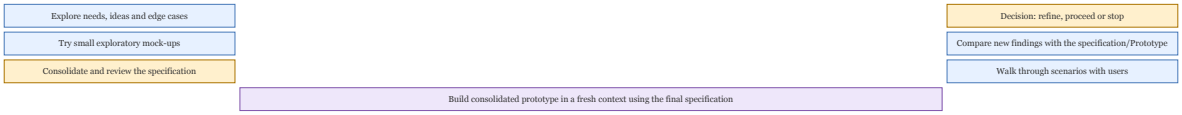


Figure 3. Own representation of a V-shaped prototype exploration cycle, inspired by the requirement–evaluation relationship in Agile V (Koch & Wellbrock, 2026).

On the left side, the citizen developer explores the business problem with an AI assistant, such as ChatGPT, and relevant stakeholders. The feature set does not need to be known in advance. Exploration may involve discussing requirements, questioning assumptions, identifying edge cases and generating alternative designs. Small, disposable mock-ups may be created to investigate individual screens or interactions. These exploratory outputs help clarify the solution but are not automatically treated as agreed requirements.

Before building the consolidated prototype, the citizen developer converts the findings into a concise, standalone specification. It identifies the intended users, selected features, main user journeys, relevant edge cases, expected outcomes, exclusions and any simulated behaviour. Contradictory or outdated ideas are removed, and unresolved questions are recorded explicitly. The citizen developer reviews the specification for consistency and business relevance. A more experienced colleague or relevant specialist is involved where uncertainty warrants it.

At the bottom of the V, the consolidated prototype is built in a fresh chat or project context using the reviewed specification and deliberately selected reference material. A tool such as Claude Design, Figma or another prototyping environment may support this activity. The exploratory conversation is not carried forward as the authoritative build instruction. Instead, the specification provides a clear baseline for what should be demonstrated. The work remains within the approved experiment boundaries, including permitted tools and data.

On the right side, the citizen developer walks through the specified scenarios with intended users or

relevant business stakeholders. Findings are compared with the expected outcomes, and a short record captures useful elements, shortcomings and unanswered questions. If feedback changes the intended behaviour, the specification is updated before the prototype is revised. Minor corrections can continue within the build context. Substantial changes return to specification review and may justify a new build context. These return paths are omitted from the figure for readability.

5.2.3.3 Build and verify

5.2.3.3.1 General Best Practices

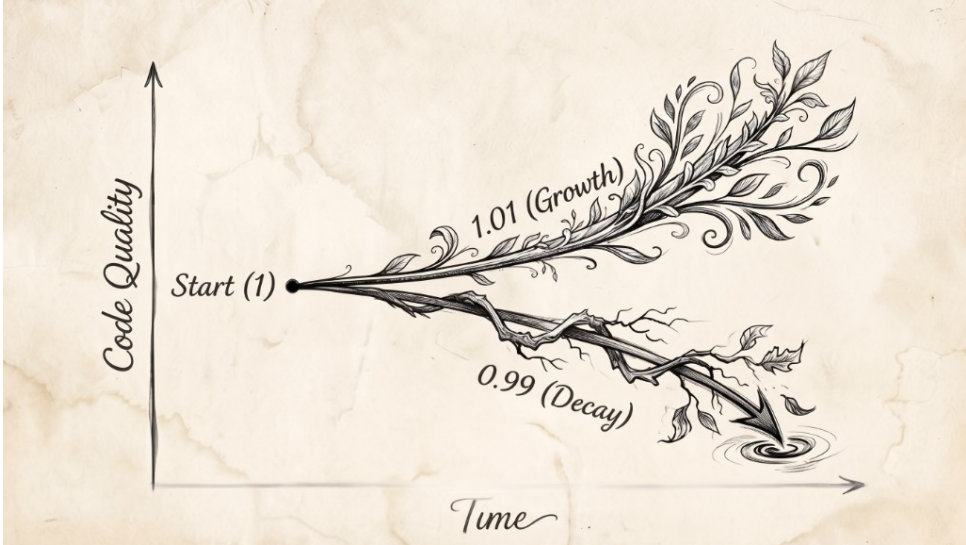


Figure 4. Conceptual illustration of compounding correctness and compounding error in AI-assisted development (own illustration).

Without structural guardrails, unstructured vibe coding can create a negative feedback loop. Initial defects, unstated assumptions and hallucinated logic may be carried forward and build on each other across multi-turn interactions, making subsequent changes harder. Sarkar and Drosos (2025) describe "context momentum": earlier prompts and generated outputs influence later development and can make it difficult to move away from initial implementation choices. In their example, the model interpreted a request for historical exchange rates as a single date. When the developer later explicitly requested a date range, the newly generated code still fetched only one date. If such choices are made directly in the production build, they may become fixed before the requirements are fully understood. A related concern is epistemological debt, which Ginac (n.d.) defines as the gap between a system's behaviour and its maintainers' understanding of it. This conceptual framework suggests that accepting generated changes without maintaining that understanding can weaken the ability to diagnose and correct failures. It does not establish that errors inevitably multiply with every interaction. Prototyping can partly address both concerns: implementation choices are explored before the production build, and citizen developers can understand the intended behaviour before production code is generated.

When vibe coding is bounded by a technical harness, explicit specifications and verification against ground truth, later iterations can build on validated results. Correctness and quality may then accumulate over time. A technical harness is a configured environment of tools, constraints and automated checks. A validated prototype allows production development to start from a reviewed basis. When the specification derived from the prototype is reviewed, changes are checked against explicit expected behaviour, and validated corrections are retained in tests and rules, later iterations can build on earlier improvements. In an industry experience report, Lopopolo (2026) describes using enforceable architectural constraints, repository documentation and automated feedback to guide coding agents. Similarly, S. Wang (2026) proposes developer-validated contracts that connect intended behaviour with code generation, testing and verification. VibeContract is a vision paper supported by an illustrative example, rather than broad empirical validation. In the proposed framework, the validated prototype is the source of this intended behaviour. Together, these sources motivate the possibility of cumulative quality

improvements. They do not guarantee that every iteration increases correctness.

5.2.3.3.2 The Agile V Development Framework

In the production-development stage, the framework uses a more complete version of Agile V than in the prototype stage. Each feature or larger change now goes through its own small V-cycle. This cycle includes independent testing, traceability (a clear link between each requirement and its tests) and documented evidence. It combines two elements from earlier work. From Agile V, it takes the idea that separate AI agents build the software, write the tests, run the tests and collect the evidence (Koch & Wellbrock, 2026). From Agentic Agile-V, it takes the six-step SCOPE-V task cycle (Koch, 2026). These are complemented by spec-driven development, contracts and practitioner approaches to planning, which are introduced below. How these elements are combined, assigned to agents and placed within the stage-gate lifecycle is a design choice of this thesis. It is important to note that the framework builds upon the technical and governance setup provided by the company explained in Section TBD!!.

The cycle is based on three principles. The first is spec-driven development (SDD). Every meaningful change starts from a written specification that is stored and versioned together with the code. This specification acts as the contract between people and AI agents. Code, tests and documentation are derived from it. If the specification and the code disagree, the specification counts as correct (Díaz et al., 2026). Furthermore, the specification documentation are complemented by a shared overall plan for the overall change of the agile V cycle (Lopopolo, 2026). The plan, kept in a file such as plan.md, is created by a Planning Agent from the specification once it has been approved at Checkpoint A. The third principle is contract- and test-based verification. A change is accepted because independent tests and other evidence show that it matches the specification, not because the result looks convincing (Koch, 2026). The V-shape links the principles: the left side creates the specification, the bottom builds the change, and the right side checks that the change matches the specification and all the updates are then saved in the plan.md. Most sources behind these ideas are conceptual or based on practitioner experience, and they have not been tested with non-technical users. Their suitability is therefore evaluated through ADR.

As shown in the prototype stage, non-technical participants can judge whether software looks and behaves as intended. A stricter structure is needed at this stage because what has to be checked changes. They cannot judge whether the code behind it is correct, secure or easy to maintain. In other words, citizen developers can mainly check functional requirements, that is, what the software does. Especially with specification-driven development (SDD), a clear specification allows reviewers to check whether the evidence demonstrates that the requirements have been met, instead of trying to work out the intended behaviour from the code (Díaz et al., 2026). Still most non-functional requirements, such as security, data protection, reliability and maintainability, are hidden in the code and lie beyond their judgement. Checking these qualities must therefore be shifted as far as possible to other means: the technical setup provided by the company and department, such as approved platforms and automated checks; AI agents that test and analyse the code; and, where needed, supporting stakeholders such as IT, information security or data protection.

To support the with non-functional requirements, Agile V gives two different jobs to two different agents. A Test Designer writes tests based only on the requirements and never sees the code, while a separate Build Agent writes the code. This separation is meant to prevent tests that simply confirm what the code happens to do instead of what it should do (Koch & Wellbrock, 2026). Confirmed behaviour is kept in tests that every later change must also pass. In this way, quality can build up over time, which is how the framework aims at compounding correctness (see Figure 4).

The cycle operates on three nested levels. In Agentic Agile-V, Agile V steers the project as a whole, while SCOPE-V steers the individual tasks carried out by agents (Koch, 2026). In this thesis, the stage-gate lifecycle takes over the steering of the application as a whole, including investment, ownership, pilot and release. Agile V moves one level down and steers how each single change is specified, built and accepted during the engineering and verification stage. SCOPE-V remains at the lowest level and steers how the Build Agent carries out each small task within a change. For example, in an HR leave-request tool, the stage-gate lifecycle decides whether the tool may be piloted, a V-cycle handles the feature for submitting a request, and SCOPE-V handles single tasks within it, such as ensuring that the leave balance cannot become negative. A V-cycle therefore ends when a tested change is accepted, not when the application is released; the release remains a decision of the stage-gate lifecycle (see Figure 5).

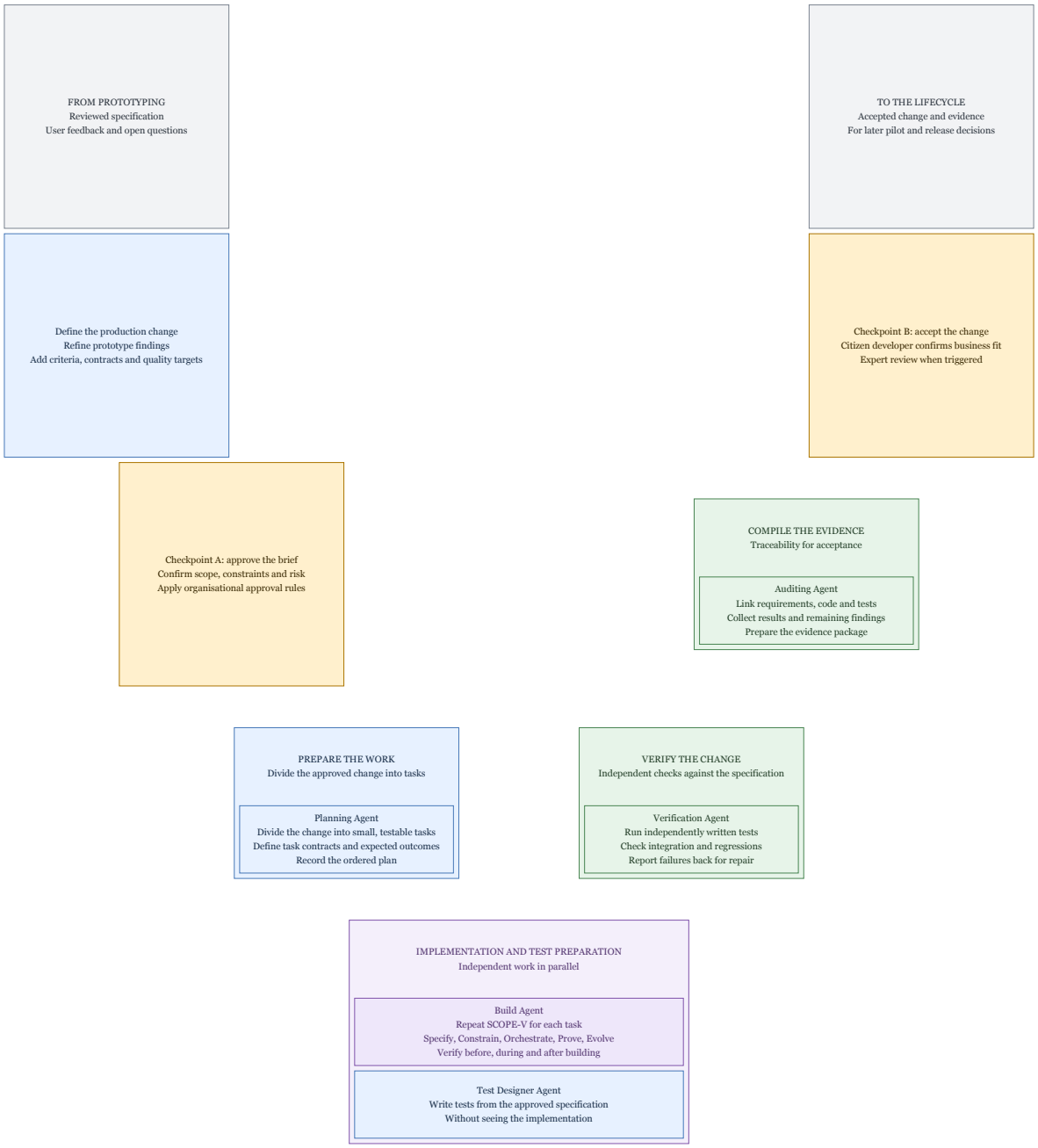


Figure 5. Own representation of an adapted Agile-V development cycle with SCOPE-V task execution (Koch, 2026; Koch & Wellbrock, 2026). Its integration into the stage-gate lifecycle and the distinction between change acceptance and production release are proposed in this thesis.

5.2.3.3.3 Left side: specifying the change

Following SDD, the specification is divided into two levels with different purposes (Díaz et al., 2026). The system specification describes the stable environment of the application: architecture rules, coding standards and other rules that every change must respect. It is stored in a rule file, such as AGENTS.md, which the agents read at the start of every session. The feature specification describes only the change currently being built within the Agile V cycle, including its acceptance criteria. Díaz et al. (2026) compare these two levels to the difference between organisational policy and operational work instructions. Keeping them apart matters: if an agent only receives the feature specification, it may produce code that works but ignores the general rules; if it only receives the system specification, it does not know what to build (Díaz et al., 2026). In their model, system specifications are typically owned by technical leads and architects, whereas feature specifications are owned by the developer responsible for the change.

This thesis adapts this division to citizen development: IT maintains the non-functional system specification, while citizen developers write the feature specification for their functional changes. As a result, citizen developers only need to describe their business need, while the technical and data-protection rules defined once by IT are given to the agents in every session. Some non-functional requirements, however, depend on the specific feature and its business context and are therefore part of the feature specification. Rules in the system specification only guide the agents. Practitioners who wrote their security rules into such a file report that agents can ignore or override instructions, so deterministic checks must still stop any violation (**TODO-koul2026vibesec**). Each rule is therefore paired with an automated check in a pipeline that IT configures once and that runs whenever a new version of the code is saved (Section 5.1.1.1).

The citizen developer is responsible for the feature specification. Starting from the tested prototype and its specification, they describe the business need, the intended users and the expected behaviour. Requirements are written as acceptance criteria in the form Given–When–Then, so that each criterion can be turned directly into a test (Díaz et al., 2026). Regarding the non-functional requiements specific to the feature, AI-generated requirements often leave out non-functional aspects. Quality requirements such as response time, security or the handling of personal data are therefore written down as short quality scenarios with a measurable target (Waseem et al., 2025). In addition, task-level contracts make hidden business rules explicit (S. Wang, 2026). Preconditions state what must be true before an action, postconditions what must be true after it, and invariants what must always remain true, for example that a remaining leave balance can never be negative. An AI assistant can draft these criteria, scenarios and contracts from the prototype specification. The citizen developer then checks them, because they describe business rules and expectations rather than technical details. Technical thresholds are confirmed with IT where needed. The resulting brief also states the goal, what is out of scope, which parts of the application are affected, which evidence is required and the risk class (Koch, 2026).

At Checkpoint A, the scope, constraints and risk class are confirmed. Checkpoint A therefore works as the conversation-to-contract gate proposed by Koch (2026): agents build from the reviewed specification, not from the earlier chat. So that citizen developers do not approve their own risk level, the risk class is set by predefined organisational rules rather than by their judgement alone. Routine changes within agreed limits continue under delegated approval rules. Some changes require technical experts: those that bring new risks, affect security-sensitive functions or personal data, have unclear technical requirements or go beyond agreed limits. If later planning reveals new risks, missing requirements or contradictions, the change returns to Checkpoint A.

As mentioned earlier, the specification documents are complemented by a shared plan for each change in the agile V cycle. The plan, kept in a separate file such as plan.md, is created by a Planning Agent from the specification once it has been approved at Checkpoint A. The agent divides the approved change into small, testable tasks, links each task to its requirements and arranges them as an ordered checklist. For each task, it defines a task contract based on the approved specification, the expected observable outcome and the requirements that must be met before moving to the next task. The Planning Agent does not create the tests; the next agent turns these requirements into tests. It does not add new behaviour beyond the approved specification. Both humans and agents update the plan. After each task is implemented, they record relevant notes and implementation decisions and update the checklist to show progress. This follows the practice of an OpenAI engineering team that built a product entirely with coding agents. The team treated plans as first-class artefacts and captured larger pieces of work in execution plans with progress and decision logs, stored and versioned together with the code (Lopopolo, 2026). The accompanying guidance describes such plans as documents that are updated at every stop-

ping point so that the work can be resumed from the plan alone (Friel, 2025).

In this framework, agents record progress and implementation decisions in the plan rather than changing the approved specification. Changes to the specification must go through Checkpoint A. The plan also helps new sessions or agent tools continue the work, reducing the need for the citizen developer to explain it again. This corresponds to the persistent shared knowledge described in SDD, which different agents and tools can read and update (Díaz et al., 2026). If a handover to the IT department is needed because the work has become too complicated, the plan provides a record of progress and decisions, while version control preserves the history of project files.

The plan consists of two parts. The first part is written in plain language and contains the purpose of the change, the tasks with their task contracts and expected outcomes, the progress, the decisions taken and their reasons, unexpected findings and open questions. The second part contains technical notes for the Build Agent, such as the files and functions to be changed. Before the build starts, the citizen developer reviews the first part, which allows them to check the intended approach before a longer implementation begins (Friel, 2025).

The plan in this framework differs from these practitioner sources in three ways (Friel, 2025; Lopopolo, 2026). First, it is kept separate from the feature specification. In the original approach, the plan itself serves as the specification. Here, the specification is the approved contract and may only be changed through Checkpoint A, while the plan changes constantly. Second, the original approach instructs agents to resolve ambiguities on their own. In this framework, ambiguities that affect requirements are recorded as open questions and returned to the citizen developer or, where needed, to a specialist. Third, the plan is created by a dedicated Planning Agent, which organises the work but does not write code. This follows the distinction in SDD between a coordinating role and agents that carry out tasks in their own contexts (Díaz et al., 2026). In the original approach, one agent writes and maintains its own plan. Here, the plan is a shared working document: the Build, Verification and Auditing Agents record their progress, results and findings in it, and the citizen developer adds decisions and answers to open questions. Each agent writes only to the parts of the plan that its role allows. Unlike the coordinating agent in Agile V, which assigns work to the specialised agents (Koch & Wellbrock, 2026), the Planning Agent does not instruct the other agents; the reasons for this are explained below. These adaptations are design choices of this thesis. The documents and agents are explained below, and Table 9 gives an overview.

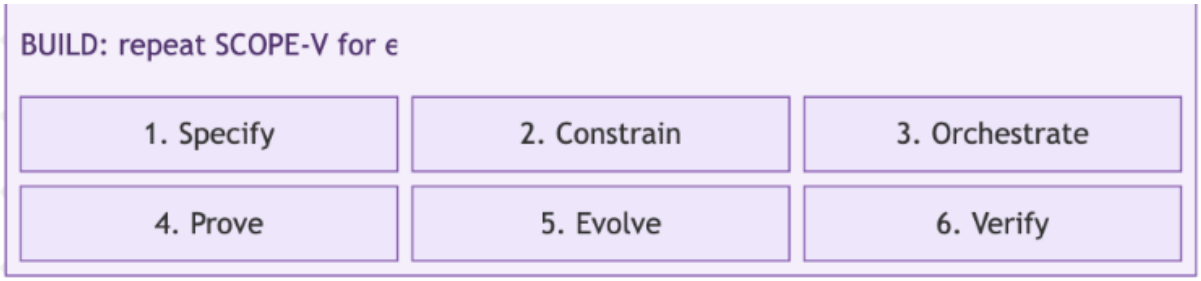
Table 9. Documents and agents in the adapted Agile V cycle (own representation).

Document	**Purpose**	**Owner / timing**
System specification (e.g. AGENTS.md)	Stable rules every change must respect, such as architecture, coding and data-protection rules	IT; read in every session
Feature specification	One change cycle: requirements, acceptance criteria, quality scenarios and contracts	Citizen developer; approved at Checkpoint A
Plan	Tasks, progress, decisions and open questions of one change	Created by the Planning Agent; updated by all agents and the citizen developer; from Checkpoint A to B
Evidence bundle	Test results, traceability matrix, scan results and a plain-language summary	Auditing Agent; before Checkpoint B
Agent	**Task**	**Does not**
Planning Agent	Splits the change into tasks and maintains the plan	Write code or add requirements
Build Agent	Builds each task through SCOPE-V	Change the independent tests
Test Designer Agent	Writes tests from the specification and task contracts	See the code
Verification Agent	Runs the independent tests	Fix code
Review Agent	Reads the change and comments on security, errors and maintainability; started by the pipeline	Change code or approve the change
Auditing Agent	Compiles the evidence bundle	Decide on acceptance

5.2.3.3.4 Bottom: building the change

During the build, the plan is kept up to date after each task, either by the Planning Agent based on the reports of the Build Agent Agent, or directly by the Build Agents. When the change is complete, the Agents adds a short summary of the outcome and the lessons learned. Decisions recorded in the plan become part of the evidence bundle at Checkpoint B, and recurring lessons are moved into the system specification through the Evolve step. Completed plans are archived with the code, so that later changes can build on them. Both practitioner sources describe the work of experienced engineers and not citizen developers (Friel, 2025; Lopopolo, 2026). Whether this approach is usable for citizen developers is therefore evaluated through ADR.

The Build Agent and the Test Designer Agent then do their work separately and in parallel (Koch & Wellbrock, 2026). The Build Agent carries out the tasks in the order set in the plan. The Test Designer Agent writes tests from the approved specification and from the task contracts and expected outcomes in the first part of the plan. It has no access to the generated code or to the technical part of the plan, since both contain implementation choices. As in the prototype stage, separate working contexts are used on purpose: agents that check the work do not share the working memory of the Build Agent. Koch (2026) recommends separating building and testing agents when the risk is high. This thesis separates them for every change, because independent tests replace the routine code review that citizen developers cannot carry out. However, the separation cannot make the specification complete. If a requirement is wrong or missing, all agents may build on the same mistake. The quality of the specification at Checkpoint A therefore remains critical.



The Build Agent completes each task through the SCOPE-V cycle (Koch, 2026). In every session, it reads the system specification together with the feature specification and its current task in the plan. It carries out all six SCOPE-V steps for each task on its own:

Specify turns the task into a short brief with a goal, scope, acceptance criteria and required evidence.

Constrain sets limits, for example no changes to unrelated files and no new external libraries without a reason. Changes to security-sensitive code are flagged and handled according to the risk class of the change.

Orchestrate defines how the agent works: first examine the existing code, then plan, build in small steps and run quick checks.

Prove collects evidence that fits the risk. This can include unit tests (single parts), integration tests (parts working together), regression tests (existing functions still work), automated code analysis and security scans.

Evolve feeds confirmed lessons back into instructions, templates and tests, and removes outdated instructions. If the same correction is needed repeatedly, it is turned into a rule in the system specification after review, so that it applies to all future work (Díaz et al., 2026). Such changes to shared instructions or specifications follow the same change controls as other changes.

Verify is not a final step. Checks are repeated before, during and after building.

While working through a task, the Build Agent tests its own work. These self-checks belong to the Prove and Verify steps of SCOPE-V (Koch, 2026). They are small and fast tests for one task, and the Build Agent is in charge of them: it decides which checks are needed, writes them, runs them and repairs the code until they pass. The citizen developer does not manage these checks one by one. The Build Agent only

records in `plan.md` that a task is complete and that its checks have passed. Agentic Agile-V recommends running such targeted checks after each small change, so that testing happens inside the agent's work rather than after it (Koch, 2026). Typical self-checks are:

- **Unit tests** check the smallest parts of the code, for example a function that calculates the remaining leave balance.
- **Existing tests of the changed area** are run again to see whether the task has broken something next to it.
- **Code-form checks** (linting and type checks) automatically find obvious mistakes, such as a misspelled name or a value of the wrong kind.
- **Quick security scans** look for passwords written into the code and for patterns that IT has marked as unsafe in the system specification.

These self-checks give fast feedback and find mistakes where they arise, while they are still cheap to fix. Their limit is that they are not independent. The agent that wrote the code also decides what to test, so the tests may simply confirm what the code does instead of what it should do (Koch & Wellbrock, 2026). Coding agents also sometimes report a task as complete when it is not (Al Hasan & Biswas, 2026; Uddin, n.d.). The self-checks therefore do not count as evidence for accepting a change. They stay in the code base and are run again with later changes, but they never replace the independent tests. Within a single task, these self-checks are sufficient. Independence is ensured at the level of the whole change, where separate agents design and run the tests (Section 5.2.3.3.5). Table 10 summarises the difference.

Table 10. Self-checks of the Build Agent compared with the independent tests of a change (own representation).

	Self-checks (SCOPE-V)	**Independent tests (Testing and verification)**
Purpose	Keep each task working before the next one starts	Show that the whole change meets its specification
Written by	Build Agent	Test Designer Agent
Based on	The task and the code the agent has just written	The approved feature specification and the task contracts, without seeing the code
Run by	Build Agent	Verification Agent and the automated pipeline
When	During each task	After all tasks of the change are complete
Who is in charge	Build Agent	Citizen developer starts the agents; IT reviews the results
Evidence at Checkpoint B	No	Yes

The other agents therefore work on the change as a whole, not on each task. The Test Designer Agent writes its tests once per change, based on the specification and the task contracts, while the Build Agent is building. When the Build Agent has finished all tasks, the Verification Agent runs these tests and the Auditing Agent then collects the evidence on the right side of the agile V framework. This matches the Agile V case study, where one cycle covered a whole piece of work with several requirements, not a single small task (Koch & Wellbrock, 2026).

This set-up has an advantage and a drawback. Only one agent works on each task, which keeps cost and effort low, and the independent checks happen where the change is accepted. However, mistakes that the Build Agent's own checks miss are only found after several tasks. For high-risk changes, the independent tests are therefore run after each task or after each milestone in the plan. This way of dividing the work between the agents is a design choice of this thesis. The test agent and the verification process will be explained in more detail in Section TBD, as it is one of the most critical steps in vibe coding (BCG Platinion, 2026).

The agents do not talk to each other directly. Instead, they coordinate through shared, versioned files: the specification, the plan, the tests and the test reports. Each agent starts in a fresh session, reads only

the files its role allows and writes its results back to these files. The order in which the agents work is fixed by the V-cycle (planning, then building and test design, then verification, then auditing). No agent decides this order. The citizen developer starts each step with a prompt that they have reviewed. The Planning Agent can suggest these prompts or check prompts the citizen developer has written. Each prompt mainly states which role the agent takes, which files it should read and what result it should produce. Because the prompts are concise, the citizen developer can understand them fully and review them quickly. They then check the outcome of each step before the next step begins.

The citizen developer therefore acts as the accountable owner of the change rather than as a programmer. They define the intent, approve the specification at Checkpoint A, answer open questions, adjust the plan where needed and accept the evidence at Checkpoint B. What they verify themselves is limited to what they can judge: functional behaviour and business rules. Non-functional qualities remain with automated checks, the testing agents and, where needed, IT specialists (see Section TBD).

This human-in-the-loop design is a deliberate choice. Vision papers propose fully automated development lifecycles in which an orchestrating meta-agent steers all phases, including verification, with little or no human involvement (Maes, 2026). This thesis does not follow this direction, for three reasons. First, practitioner reports describe agents that complete roughly 70% of a task quickly but struggle with the remaining integration and edge cases, often ending in repeated fix loops (Osmani, 2024; Režun, 2026). Second, studies of coding agents report constraint violations, unintended changes to the environment and false claims that a task is complete (Al Hasan & Biswas, 2026; Uddin, n.d.). Third, without human involvement, no one in the organisation understands the application well enough to take responsibility for it, maintain it or hand it over (Ginac, n.d.; Mitchell et al., 2026). For applications used in productive environments this last point is decisive. Accountability must stay with a person like the citizen developer, while technical accountability is delegated to specialists.

The same reasoning applies to how the agents themselves are coordinated. In Agile V, a coordinating agent assigns work to the specialised agents and tells them which files to read, so that each starts with a fresh context (Koch & Wellbrock, 2026). This thesis deliberately limits that role. The Planning Agent plans the change, keeps plan.md up to date and explains progress, decisions and problems to the citizen developer in non-technical language. However, it does not start or instruct the Build, Test Designer, Verification or Auditing Agents. There are four reasons for this. First, transparency: if agents manage other agents, several layers of abstraction separate the citizen developer from the work, which makes it harder for them to see what is happening and where the change stands. Second, independence: an agent that sees the code and also briefs the Test Designer could pass on implementation details. This would weaken the separation between building and testing, which replaces the code review that citizen developers cannot carry out. Third, traceability: when a person starts each step and the result is recorded in the shared files, every decision can be linked to a responsible person and to evidence. Fourth, collaboration: the plan becomes the single place where the citizen developer follows and steers the work. The approved specification stays stable, so plan.md is the only document that changes continuously during the build. Every agent reports its results there in short, plain language: completed tasks, test results, audit findings, decisions and open questions. The citizen developer therefore does not need to follow several chat sessions, remember what each agent was told or rely on an agent's own claim that a task is complete. Such claims are not always reliable (Al Hasan & Biswas, 2026). Instead, they read one document, keep it correct and react to its open questions. Each entry links to the underlying test or audit report, so progress can be checked against evidence rather than taken on trust. Because the plan is kept up to date at every stopping point, the work can also be resumed in a new session, or handed over to IT or other specialists, from this document alone (Friel, 2025). The plan thus serves as a shared reference point not only between the citizen developer and the agents, but also between the citizen developer and the experts who support them.

Separate contexts remain important, because model performance tends to decline as the input grows longer and mixes unrelated information (Hong et al., 2025). The framework achieves this through a fresh session for each role rather than through an orchestrating agent. The Planning Agent may still use short-lived sub-agents for read-only tasks, such as summarising code or answering a question from the citizen developer. This keeps its own planning context free of technical detail (Anthropic, 2025). These sub-agents do not change any files. The trade-off is that the citizen developer has to carry out more steps manually. The ADR cycles evaluate whether this extra complexity is acceptable (Section TBD).

5.2.3.3.5 Right side: verifying and accepting the change

Testing decides whether AI-generated work is accepted. Professional developers usually check code in two ways: they run tests and they review the code itself. Citizen developers cannot read the code, so additional tests and automated checks take the place of this code review. A change is accepted because the evidence shows that it meets its specification, not because it looks plausible (Koch, 2026). This part deals only with the *independent* tests of a change: the tests that the Test Designer Agent writes and the Verification Agent runs after the Build Agent has finished all tasks. It does not cover the small self-checks that the Build Agent runs within SCOPE-V while it builds; these are described in Section 5.2.3.3.4 and do not count as evidence for acceptance. Testing of the whole application follows later, in stage 4 (Section 5.2.3.4).

A separate Verification Agent, which corresponds to the Red Team Verifier in Agile V (Koch & Wellbrock, 2026) [1], runs the independently written tests on the combined software. This differs from the Verify step of SCOPE-V. There, the Build Agent checks its own work; here, the Verification Agent tests the result independently. Tests on single tasks help find errors in individual changes.

The Test Designer Agent writes its tests once per change, from the approved feature specification. It works in parallel with the Build Agent, so testing does not delay the build. In Agile V, this separation prevents tests that only confirm what the code happens to do instead of what it should do (Koch & Wellbrock, 2026). Table 11 lists the tests used, which the citizen developer can explicitly ask for in the Test Designer Agent prompt. Besides the agent's tests, the automated pipeline runs the scans defined by IT. The automated pipeline is a set of checks that IT configures once and that runs automatically whenever a new version of the code is saved. The scans are included here because they are part of the evidence at Checkpoint B.

[1] In Agile V, "red team" refers to independent verification: the Red Team Verifier runs tests that were written without access to the code and reports which requirements pass or fail (Koch & Wellbrock, 2026). It does not mean security red teaming, in which the behaviour of an attacker is simulated. Adversarial security testing is treated separately in this framework (Sections 5.2.3.3.5 and 5.2.3.4).

Table 11. Independent tests and checks of a change (own representation).

Test	What it checks	Example (leave-request tool)	Created by
Acceptance tests	Each acceptance criterion of the specification, written as Given–When–Then	Given an employee with ten days left, when they request three days, then seven days remain	Test Designer Agent
Contract tests	Rules that must hold for each step: *Preconditions*: what must be true before the step runs; *Postconditions*: what must be true after the step; *Invariants*: what must always be true	*Precondition*: a request is only accepted if the start date lies before the end date; *Postcondition*: after approval, the balance is reduced by exactly the number of requested working days; *Invariant*: the leave balance never becomes negative	Test Designer Agent
Negative and boundary tests	Wrong inputs, limits and forbidden actions	A request for more days than available is rejected; an employee cannot open the approval page	Test Designer Agent
Integration tests	Whether the change works with neighbouring parts	An approved request is saved correctly in the database	Test Designer Agent
Regression tests	Whether functions accepted in earlier changes still work	Submitting and cancelling requests still work after the change	Test Designer Agent in earlier cycles
Library allow-list check	Whether every external library is on the list approved by IT	A package suggested by the agent that is not on the list is blocked	Rules set by IT; run by the pipeline
Static analysis and security scans	Known weaknesses in the code, passwords in the code and unsafe external libraries	A database query built directly from user input is flagged	Rules set by IT; run by the pipeline
AI code review	Problems in the code that no requirement mentions, such as unsafe defaults or hard-to-maintain code; advisory only	A password written into a configuration file is flagged for the IT reviewer	Review Agent; configured by IT

Security is tested in two ways at this level. The first way checks the code against general rules that IT defines once in the system specification and in the pipeline. *Static analysis* reads the code without running it and looks for known unsafe patterns. *Secret scanning* searches for passwords or access keys written into the code. A *dependency scan* checks whether the external libraries used by the application have known security problems. Agentic Agile-V lists such static and security checks as required evidence for production changes (Koch, 2026). IT bases these rules on an established catalogue of common weaknesses in web applications, such as the OWASP Top 10 (**TODO-owasptop10**). Sapkota et al. (2025) describe automated security auditing against this catalogue as a typical task for coding agents: an agent runs static analysis tools, applies fixes and writes a security report. This framework uses the catalogue as a checklist for the rules and scans, but it does not let one agent both find and fix weaknesses, because this would combine checking and building in one role. Findings are returned to the Build Agent like any other failed test. The example in Sapkota et al. (2025) is an illustrative use case in a review, not an evaluated one.

The Build Agent runs quick versions of these scans during its tasks (Section 5.2.3.3.4); here, the pipeline runs them in full for the whole change. The second way covers security behaviour that depends on the feature, such as which user role may approve a leave request. This behaviour is part of the feature specification, so the Test Designer Agent turns it into acceptance and negative tests, for example: "Given a user with the role employee, when this user opens the approval page, then access is denied."

Protection against invented and unsafe libraries. A third risk concerns the external libraries that an agent adds to the application. Language models sometimes recommend software packages that do not exist. **TODO-huang2026package<empty citation>** collected 107 such invented package names from eight models and registered them in the public npm registry. Over six months, about a third of the installations were attributed to developers rather than to automated scripts, and a quar-

ter of the names differed from an existing package by at most one character. In their setting, giving the models web search did not solve the problem but made the same invented names appear more often (**TODO-huang2026package**). An attacker who registers such a name can deliver harmful code to anyone who follows the recommendation. A dependency scan may not detect this, because it mainly reports known weaknesses of packages that already have a security record. **TODO-huang2026package<empty citation>** therefore propose NymGuard, an agent that checks recommended names against the registry, registers invented names in advance and publishes warnings. NymGuard protects the public registry and is not designed for use inside a company. The authors also note that checking whether a name exists is not sufficient on its own, because an attacker may already have registered it (**TODO-huang2026package**). This framework therefore adopts the underlying idea in a simpler form. IT keeps a list of approved libraries in the system specification, and the pipeline blocks any change that adds a library outside this list. A request for a new library goes to IT for review. This extends the Constrain step of SCOPE-V, which allows new external libraries only with a reason, and follows the practitioner advice to use only well-established libraries and to audit every dependency (**TODO-koul2026vibesec**). The allow-list is a design choice of this thesis; its effort for IT and its effect on citizen developers have not been evaluated.

Order of the tests. After the Build Agent has completed all tasks, the Verification Agent runs the independent tests in four steps, from narrow to wide and from fast to slow, so that a failure can be traced to its cause:

1. **Static analysis and security scans.** They are fast and do not need a running application. Clear rule violations are stopped before more expensive tests start.
2. **Acceptance, contract, negative and boundary tests.** They check whether the new behaviour matches the specification. A failure here points to the change itself.
3. **Integration tests.** They check whether the change works together with neighbouring parts.
4. **Full regression suite.** It checks whether earlier functions still work. It runs last because it covers the most ground; a failure here shows that the change has side effects on other features.

This order follows the guidance of Agentic Agile-V to run continuous integration, static analysis and security checks before a change is merged (Koch, 2026). The exact sequence is a design choice of this thesis.

Repairs and protection of the tests. If tests fail, the error reports are sent back to the Build Agent, which fixes the code and submits it again for testing. This feedback loop follows Agile V, where the Build Agent fixed the issues found by the verifier in a rework round (Koch & Wellbrock, 2026). It also follows the contract-based feedback loop of VibeContract (S. Wang, 2026). To avoid endless repair attempts, the number of automatic repair rounds is limited. As one example, **TODO-tddgovernance<empty citation>** allow three repair attempts and stop earlier when the same failure repeats or a repair changes nothing. If the change still fails after this limit, it is recorded in the plan as an unresolved finding and passed to the citizen developer, who can research the problem to solve it or pass it to an expert. The exact limit is a design choice of the organisation. The tests themselves are also protected. Coding agents tend to optimise for passing checks rather than for working software; they may change test conditions or write extra scripts that force a test to pass (**TODO-kitchenloop**). The Build Agent may therefore not change the independent tests. Every change to a test file is visible in version control and requires approval by the IT reviewer. This rule is a design choice of this thesis.

Automated review of the code. The independent tests check what the specification describes. They do not read the code, so they miss problems that no requirement mentions, such as a password in a configuration file, an unsafe default setting or code that is hard to maintain. Professional teams find such problems in code review, which citizen developers cannot carry out. The framework therefore adds a Review Agent that reads the change as a human reviewer would. **TODO-kumar2025intuition<empty citation>** describe such a system in a company with about 300 engineers: for every pull request, several agents review the change in parallel, each with its own focus, such as security, error detection, maintainability, performance and business logic. Their comments are combined, duplicates are removed and each comment names the agent that wrote it. Over one year, the time needed to review and close a pull request fell by 31.8%, and most engineers wanted to keep the review agents. Human reviewers, however, still examined both the code and the agents' comments before a change was merged, and a feature that accepted AI suggestions automatically caused quality problems and was switched off (**TODO-kumar2025intuition**).

The study measured productivity and satisfaction, not how many security problems the agents found or missed, and its participants were professional engineers, not citizen developers. Waseem et al. (2025) likewise report that some teams use AI reviewers in their pipelines, and recommend treating AI-generated code as a first draft that must pass strict automated gates. In this framework, the Review Agent is therefore advisory. The pipeline starts it, not the citizen developer; it does not change code; and its comments are added to the evidence bundle. At Checkpoint B, they help the IT reviewer decide where to look more closely, but they do not replace the technical approval. This role is a design choice of this thesis.

Responsibilities for the tests. The citizen developer manages the independent testing: they start the Test Designer and Verification Agents with reviewed prompts and follow the results in `plan.md`. IT defines the security and quality rules, configures the pipeline and the scanning tools and judges their findings.

Evidence bundle. An Auditing Agent, which corresponds to the Compliance Auditor in Agile V, then collects all evidence in one package. This package contains a traceability matrix, a table that links each requirement to the code and the tests that checked it (Koch & Wellbrock, 2026). It also contains the test results, the results of automated code analysis and security scans, important decisions and unresolved findings, as well as known remaining risks and, for production changes, a plan for undoing the change (Koch, 2026). The Auditing Agent also adds a short plain-language summary of this evidence to `plan.md`, so that the citizen developer can follow the result in the same document they use throughout the change.

Checkpoint B. At Checkpoint B, the change is accepted in two steps. First, the citizen developer decides whether the change meets the business need. This decision is based on the evidence package, not on reading the code. Second, a technical expert approves the change. Only throwaway prototypes are exempt from this technical approval. The risk class of a change does not decide whether a technical review takes place, but how deep it goes and which evidence it requires. If no expert is available, the change remains restricted or on hold and is not treated as verified.

This follows both Agile V sources, which keep technical approval in the loop. In Agile V, the responsible engineer holds final approval authority at the second human gate (Koch & Wellbrock, 2026). Koch (2026) defines four risk levels, each with stricter requirements for evidence and approval:

Table 12. Risk levels in Agentic Agile-V (Koch, 2026).

Risk level	Typical changes	Approval requirement
R0 exploratory	Throwaway prototypes and internal demos	Review optional
R1 routine	Documentation and small code clean-ups	Normal code review
R2 production	New features and bug fixes	Mandatory reviewer approval
R3 high assurance	Security-related functions, for example	Explicit sign-off and release gate

The prototype stage of this framework corresponds to R0. Most changes made by citizen developers are expected to fall under R2 and therefore require mandatory approval by a technical reviewer.

The framework does not remove this review but changes its form. Instead of reading every line of code, the expert checks whether the evidence package shows that the change meets its specification and looks more closely only where the evidence is weak. Díaz et al. (2026) describe this shift as evidence-backed acceptance and argue that it makes review faster and more informative. The depth of the review is set explicitly for each type of change and can be revised as experience grows (Díaz et al., 2026). Which types of change belong to which risk level must be defined for SIXT HR rather than taken over directly from these sources. The split between business acceptance by the citizen developer and technical approval by the expert is a design choice of this thesis. It aims to keep technical control while reducing the effort that each change demands from scarce experts.

Return paths and release. Errors in the code send the work back to the task cycle. Major changes to requirements or constraints send it back to Checkpoint A. For readability, the figure leaves out these return paths. An accepted change may be merged into the main code according to the repository rules, but acceptance does not mean release to production. Instead, the evidence from successive V-cycles feeds into the pilot and release decisions of the lifecycle, together with checks of the whole application and arrangements for operation. The same cycle is used again for changes during the pilot and in later operation.

Limits of testing a change. Independent tests of a change have four limits. First, all of them derive from the same feature specification. If a requirement is wrong or missing, the tests confirm the mistake instead of finding it. Second, other systems are often replaced by mocks, which may hide problems with the real connections. Third, the tests focus on one change, not on complete user tasks that cross several features. Fourth, passing tests show that the specified requirements are covered, not that the software is complete or free of errors. The Agile V case study, for example, reported a complete pass rate at the requirement level but did not use mutation testing, branch coverage or fuzz testing (Koch & Wellbrock, 2026). The Kitchen Loop reaches a similar conclusion: tests of single functions and interfaces are necessary but not sufficient, because they do not exercise the real execution environment (**TODO-kitchenloop**). Checkpoint B therefore accepts a change, but it does not show that the application is ready for users. This is the task of stage 4. Table 13 summarises the difference between the two levels. Appendix A summarises which tests are written and run at each level (Tables 19 and 20).

Table 13. Testing a change at Checkpoint B compared with testing the whole application at gate G4 (own representation).

	Checkpoint B (stage 3)	Gate G4 (stage 4)
Main question	Does this change do what its specification says, without breaking existing functions?	Does the whole updated application still work, do all parts work together under real conditions, and are further changes needed?
Scope	One change, such as a feature or a bug fix	The whole application as one fixed version
Environment	Development sandbox; other systems may be replaced by simulated versions (mocks)	Staging environment: a copy of the production set-up with real login, real roles and test connections to other systems
Basis for expected behaviour	Feature specification and task contracts	User journeys and test cards from the prototype, success criteria from G1, rules set by IT
Who writes the tests	Test Designer Agent, from the feature specification; IT defines the scanning rules	Users and business owner write test cards (simple task lists); the Test Designer Agent turns them into automatic browser tests; IT defines the security and operational checks
Who runs the tests	Verification Agent and the automated pipeline	On staging: the Verification Agent and the automated pipeline run the automatic tests; users carry out the test cards themselves; IT and specialists carry out the expert checks
Main tests	Acceptance, contract, negative and boundary, integration with neighbouring parts, regression, static analysis and security scans	Smoke, full regression, real integration, end-to-end journeys, role and access, security of the whole application, data protection, user acceptance, operational readiness
Responsibility	Citizen developer manages the cycle; IT sets the rules and gives technical approval	Depends on the risk tier: IT owns the technical tests from R2 upwards; citizen developer and business owner own the business tests
Result	Change is merged into the main code but not released	Pilot (stage 5), or revise, restrict, hand over or stop

The ADR evaluation will test whether this design works in practice. It examines whether citizen developers can write and check the required specifications, contracts and quality scenarios; whether the evidence package allows experts to review changes with manageable effort; whether expert availability becomes a bottleneck; and how many errors come from the specification rather than from the code.

5.2.3.4 Test application

After gate G3, all planned changes have been accepted one by one. Stage 4 checks whether the application as a whole is ready for real users. It answers three questions. Does the updated application still work as

a whole? Do its parts work together under real conditions? Are further changes needed before users get access? The answers form the basis for the decision at gate G4.

Why a separate stage is needed. Accepted changes do not guarantee a working application. In the Kitchen Loop, an agent delivered a feature that passed 38 unit tests and all style checks, yet it was completely broken, because nobody had started the service and used it as a user would (**TODO-kitchenloop**). **TODO-tddev<empty citation>** argue that the correctness of a web application cannot be judged from its source files or test output alone; the application must be deployed and used through a browser. In their benchmark experiments, testing the deployed application through a browser improved results by 34 to 48 percentage points compared with no such testing (**TODO-tddev**). A second reason lies in this framework itself: all tests in stage 3 are derived from specifications written by the citizen developer (Section 5.2.3.3.5). Stage 4 adds two further sources of truth: the users, who judge whether the application supports their work, and IT, which judges the hidden qualities. MeRGE likewise treats quality assurance as a phase of its own and found that human testing remained unavoidable and that domain experts were indispensable (Fetzer et al., 2026). This finding comes from systems that contain GenAI and is transferred here with care.

Responsibility depends on risk. In stage 3, the citizen developer manages most of the Agile V cycle. In stage 4, responsibility for the technical tests moves to IT, depending on the risk tier that was set at G1 and confirmed at later gates. This shift reflects the different kind of evidence now required. Citizen developers can judge whether software looks and behaves as intended, but not whether it is secure or reliable (Kobiella et al., 2026). For low-risk applications, IT provides a standard test kit that the citizen developer can run and IT reviews. For higher risk tiers, IT takes ownership of the technical tests. The citizen developer and the business owner remain responsible for the business tests in every tier. Table 14 shows this distribution. R0 applications do not enter stage 4, because they remain prototypes.

Table 14. Responsibilities for application testing by risk tier (own representation).

Tier	Technical tests	Security tests	Business tests	G4 decision
R1 routine	Citizen developer runs the standard IT test kit; IT reviews the evidence report	Automated scans and role tests; IT reviews findings	Citizen developer with users	Business owner and IT reviewer
R2 production	IT owns the tests; agents and pipeline run them	As R1, plus scans of the running application; data protection check if personal data is processed	Citizen developer and business owner	Business owner, IT reviewer, specialists if triggered
R3 high assurance	IT leads all technical testing; the application is co-developed or handed over	As R2, plus a penetration test by IT security; data protection and employee representation involved	Business owner with users	Explicit sign-off by business owner, IT and specialists

Preparing the tests early. Several test inputs are created before stage 4 begins. At G2, the citizen developer writes the main user journeys and short test cards together with users and the business owner. A test card is a step-by-step task with an expected result, for example: “Submit a leave request for three days. Expected result: the request appears for the manager and the balance shows three days less.” The cards are based on the prototype walkthroughs and on the success criteria agreed at G1. They are written before the code exists and are not given to the Build Agent. This follows the information wall of the Kitchen Loop, in which the evaluator receives only the test card and no code or implementation context. The Kitchen Loop also requires each card to describe only behaviour that users can see and to include at least one step in which a wrong input is rejected (**TODO-kitchenloop**). Unlike in the Kitchen Loop, where the implementing agent writes the cards, here the users write them, because they know best what the application must do. The citizen developer also prepares a role matrix that states which role may see and do what. During stage 3, the regression suite grows with every accepted change. IT provides a reusable test kit: a staging environment, an automated pipeline, scanning rules, templates for role tests and a rollback procedure. Because the kit is built once and reused for every application, expert effort per application is expected to decrease. When stage 4 starts, the application is frozen as a release candidate, a fixed and labelled version in version control. During testing, only fixes are made, no

new features.

Agents in this stage. The agents from stage 3 keep their roles but work on the whole application. The Test Designer Agent turns the user journeys and test cards into executable browser tests, again without access to the code. The Verification Agent runs the regression suite and the browser tests on the staging environment. A browser test opens the application and clicks and types like a user. When a test fails, the agent writes a short report that states what was attempted, where it failed and what was visible on the screen. This follows the approach of TDDev, which converts requirements into acceptance tests, runs them in the browser and translates failures into structured reports (**TODO-tddev**). Such test agents are not fully reliable: in TDDev, the testing agent's verdict was correct in 87.5% of cases, with errors only in the form of missed passes (**TODO-tddev**). These results come from benchmark tasks and a user study with three professional developers, not with citizen developers. Screenshots are therefore kept as evidence, and failures of critical journeys are checked by a person.

Adversarial security tests. For R2 and R3 applications, a security test agent additionally attacks the running application on staging. Unlike the Verification Agent, it does not check whether the specification is met but tries to break the application. It acts from the perspective of each role and attempts forbidden actions, for example opening the leave data of another team through a direct link, changing a request that belongs to someone else or sending manipulated input to a form. Several practitioner sources describe this idea. In an experience report, Waseem et al. (2025) scanned seven early vibe-coded applications with an agentic security scanner and found 970 security issues, most of them rated as highly severe; one co-author proposes "white-hat hacker agents" that carry out controlled attacks during development and acceptance. BCG Platinion lists dedicated red-team agents that probe adversarial edge cases as one layer of verification in an agentic software factory (BCG Platinion, 2026). A Thoughtworks team that scaled a vibe-coded prototype asked the AI to act as an attacker and test what it had built, and reported that this regularly revealed weaknesses in permissions and data exposure (**TODO-koul2026vibesec**). None of these sources reports how many real weaknesses such agents find or miss. Waseem et al. (2025) therefore recommend using such agents as an additional defence and checking their findings regularly through manual penetration tests. The framework follows this advice: the agent works only on staging with test data, its findings go to IT security, and for R3 applications a penetration test by IT security remains mandatory. Confirmed findings become regression tests and, if they reveal a recurring pattern, a rule in the system specification, so that red-team findings improve the rules for later changes (BCG Platinion, 2026). Finally, the Auditing Agent compiles the G4 evidence report and adds a plain-language summary to `plan.md`. As in stage 3, no agent approves anything (Al Hasan & Biswas, 2026).

Order of the tests. Table 15 shows the order in which the tests are run.

Table 15. Order of the tests in stage 4 (own representation).

#	Test	What it checks	Run by	Reviewed by
1	Smoke test	The application starts on staging, login works, the main pages load and connected systems respond	Pipeline	IT
2	Full regression	All tests from stage 3, now in the staging environment	Verification Agent	IT
3	Real integration	Real database, login and Workday test interface instead of mocks; data are checked where they are stored, not only on the screen	Verification Agent	IT
4	End-to-end journeys	Complete user tasks that cross several features, run in the browser	Verification Agent	Citizen developer
5	Role and access tests	Every role in the role matrix: permitted actions work and forbidden actions are blocked	Verification Agent, security test agent	IT
6	Security of the whole application	Full scans of code and libraries; scans of the running application (R2 and higher); penetration test (R3)	Pipeline, IT security	IT security
7	Data protection	Which personal data are stored, logged and deleted, and where; only when triggered by the risk tier	Specialist	Specialist
8	User acceptance	Real users complete the test cards without help from the citizen developer	Users	Business owner
9	Operational readiness	Rollback tested once, monitoring and error alerts active, backup, named operations owner, user guide	IT, operations owner	IT
10	Evidence report	Summary of all results, open findings and remaining risks	Auditing Agent	Gate decision-makers

The order follows three rules. First, automated tests come before human tests. Users' and experts' time is scarce, so they should only test a version that already works technically. Second, cheap tests come before expensive ones. If the smoke test fails, there is no point in running the other tests. Third, the tests move from the inside to the outside. They first check whether earlier functions still work (step 2), then whether the parts connect to the real systems (step 3), then whether complete tasks succeed (step 4) and finally whether the application is secure, lawful and usable for real work (steps 5–8). The Kitchen Loop describes a similar layering, in which tests of real execution and complete user journeys give the highest level of trust (**TODO-kitchenloop**). Two choices adapt its principles to citizen development. User acceptance is placed late, because users judge the application best when it is technically stable. And the "weakest evaluator" principle of the Kitchen Loop, in which a deliberately weak model executes the test cards (**TODO-kitchenloop**), is translated into a human rule: test cards are also carried out by a colleague who has not seen the application before. If they can follow the card, real users can too.

Pass and fail criteria. Table 16 lists the criteria for passing stage 4. Findings are grouped as critical, major or minor, in the same way as the IT review findings in this study (Section 2.4). Exact thresholds, such as the share of test cards that users must complete, are set by the organisation at G1.

Table 16. Pass and fail criteria in stage 4 (own representation).

Check	Passed when	Blocks the gate when
Smoke and regression	All tests pass; no test was deleted or weakened without IT approval	Any test fails or was changed without approval
Unstable tests	Tests that sometimes pass and sometimes fail without a code change are repeated, then fixed or set aside with IT approval	An unstable test is counted as passed
Integration	Data are stored and changed correctly in the real systems	The screen reports success but the data are wrong
End-to-end journeys	All critical journeys pass	A critical journey fails
Role and access	No forbidden action succeeds	Any access by a role that is not permitted
Security	No open critical or high findings; medium findings have an agreed plan	An open critical or high finding
User acceptance	Users complete the critical tasks without help; the business owner accepts	Users cannot complete core tasks
Operations	Rollback works; owner, monitoring and support are in place	Any of these is missing
Traceability	Every requirement is linked to at least one passed test	A requirement remains untested

When tests fail. A failed test is first classified by its cause, because the cause decides where the work goes back to (Table 17). Each failure is recorded in `plan.md` together with its cause. This record also serves the evaluation of this thesis, because it shows how many errors stem from the specification rather than from the code.

Table 17. Handling of failed tests in stage 4 (own representation).

Cause	Return path	Responsible
Error in the code	New change in stage 3. A test that reproduces the error is written first, then the smallest possible fix (Koch, 2026)	Citizen developer and agents; IT approval
Wrong or missing requirement	Checkpoint A, or stage 2 if the change is large	Citizen developer with users
Wrong test	The test is corrected and the correction approved by someone who did not write the code	IT
Environment problem	Not counted as an error of the application. If the failure also occurs with the previous version, the cause lies in the environment (**TODO-kitchenloop**)	IT
Security or data protection problem	Fix in stage 3; otherwise restrict or hand over	IT, specialists
Repeated failure	The repair limit is reached; the problem is passed on	IT; gate decides on revise or hand over

After a fix, the smoke test, the full regression suite and all failed or affected tests are repeated. User acceptance is only repeated if the fix changes behaviour that users can see. Returns from this stage to development are expected and not a sign of failure: in MeRGE, the most frequent transitions occurred between development and quality assurance (Fetzer et al., 2026).

Gate G4. At G4, the decision-makers check whether:

- all blocking criteria in Table 16 are met;
- every requirement is linked to a passed test;
- the business owner has accepted the user acceptance results;
- the security and data protection checks required for the risk tier are completed and accepted;
- rollback, monitoring, support and an operations owner are in place;

- open minor findings are documented with an owner and a deadline.

If all criteria are met, the application moves to the pilot in stage 5 (Go). Otherwise, the gate decides to revise, restrict, hand over or stop. A restriction may, for example, allow a pilot only with fewer roles or without real personal data. A handover is appropriate when the tests show that the application exceeds what citizen development can safely deliver, for example because integrations with enterprise systems fail repeatedly. In such cases, research on low-code development recommends that IT co-develops or leads (Viljoen et al., 2024).

Testing later updates. Changes made during the pilot and operation pass through stage 3 again and then through a reduced version of stage 4. The automated tests (smoke test, full regression suite and scans) always run in full, because they are cheap. The human tests (end-to-end checks by the citizen developer, role reviews by IT and user acceptance) are repeated only for the parts of the application affected by the change. The rule is simple: automated tests always check everything; people check what has changed.

The structure of this stage, the shift of responsibility to IT by risk tier, the early test cards written by users and the order of the tests are design choices of this thesis. They build on practitioner reports and on studies with professional developers. Whether they work for citizen developers is evaluated through eADR. The evaluation examines how many errors stage 4 finds that passed Checkpoint B, how many of them stem from the specification, whether users can write usable test cards and whether the effort for IT remains manageable.

5.2.3.5 Pilot

5.2.3.6 Operate and reassess

Monitoring new vulnerabilities. An application that passed all checks at release can become insecure later, when a weakness is found in one of its libraries, in the platform or in the AI coding tools themselves. During operation, the framework therefore includes a security intelligence feed. **TODO-koul2026vibesec<emp** describe such a feed at Thoughtworks: an automated workflow monitors the languages, cloud platforms and AI coding tools that a team uses and delivers a daily digest of new vulnerabilities and security advisories, so that the team learns of a problem on the day it is published. In this framework, IT runs the feed centrally for all citizen-developed applications and matches each advisory with the libraries recorded for each application. The operations owner is informed when an application is affected. The same authors envision agents that create work items and prepare patches for known weaknesses for human review, but they present this as a future step, not as current practice (**TODO-koul2026vibesec**). The framework allows such an agent to prepare a fix, but the fix is treated like any other change: it passes through the Agile V cycle of stage 3 and the reduced version of stage 4, including technical approval at Checkpoint B (Section 5.2.3.4). Urgent fixes may follow a shortened path, but they never skip the automated tests or the technical approval. Central monitoring and the shortened path are design choices of this thesis. They aim to spare each citizen developer from following security news alone.

6 Evaluation and Findings

Table 18 shows how each difficulty is traced from its origin to the evidence used to evaluate it.

Table 18. Tracing difficulties from App 1 to the evaluation in App 3 (examples; own representation).

Difficulty in App 1	Framework response	Evidence in App 3
No working earlier version could be restored	Git training and version control	Git history; ability to roll back
Features rejected late, work discarded	Prototype and user walkthrough before building	Changes after the prototype; discarded work
Existing solutions rebuilt from scratch	Check for existing libraries	Libraries reused
High cost from experimentation and model choice	[e.g. selective model use, experiment budget at G1]	Tool spend; rework
Unclear what had been verified	Specification, independent tests, evidence in `plan.md`	Test results; IT review findings
[add further difficulties]		

6.1 Starting Point

7 Discussion

8 Conclusion

References

Al Hasan, A., & Biswas, S. (2026). What Breaks When LLMs Code? Characterizing Operational Safety Failures of Agentic Code Assistants [Stored arXiv version 3]. *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. https://doi.org/10.1145/3832783.3834393

Anthropic. (2025). *Effective context engineering for AI agents*. Anthropic. Retrieved October 29, 2026, from https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

BCG Platinion. (2026). *The agentic software factory*. BCG Platinion. Retrieved October 29, 2026, from https://www.bcgplatinion.com/insights/the-agentic-software-factory

Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, *3*(2), 77–101. https://doi.org/10.1191/1478088706qp063oa

Demirer, M., Musolff, L., & Yang, L. (2026). *Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools* (NBER Working Paper No. 35275) (Working paper; revised September 2026; not peer-reviewed). National Bureau of Economic Research. http://www.nber.org/papers/w35275

Díaz, J., Gayoso, J., Cimminio, A., & Pérez, J. (2026). Spec-driven development for agentic software engineering: Harnessing human–agent teamwork. https://doi.org/10.48550/arXiv.2609.00252

Esnaola, L. M., Ramón, H. D., & Lanzarini, L. C. (2026). Can Generative AI Bridge the Gap? A Quasi-Experimental Study of Non-programmers with AI vs. Programmers without AI. *Empirical Software Engineering*, *31*, Article 75. https://doi.org/10.1007/s10664-026-10813-7

Farrag, S. E. (2026). The productivity-reliability paradox: Specification-driven governance for AI-augmented software development. https://doi.org/10.48550/arXiv.2605.01160

Feldman, M. Q., & Anderson, C. J. (2024). Non-Expert Programmers in the Generative AI Future. *Proceedings of the 3rd Annual Meeting of the Symposium on Human-Computer Interaction for Work*. https://doi.org/10.1145/3663384.3663393

Fetzer, D., Gimpel, H., Meindl, O., & Strickmann, J. (2026). Responsible engineering of information systems based on generative artificial intelligence: An action design research study at a german premium car manufacturer. *Business & Information Systems Engineering*, *68*(3), 583–608. https://doi.org/10.1007/s12599-025-00950-6

Fortes-Ferreira, M., Alam, M. S., & Bazilinskyy, P. (2025). Vibe Coding in Practice: Building a Driving Simulator Without Expert Programming Skills. *Adjunct Proceedings of the 17th International Conference on Automotive User Interfaces and Interactive Vehicular Applications*. https://doi.org/10.1145/3744335.3758482

François, P. A., Ciftci, S. A., Janiesch, C., & Plattfaut, R. (2025). Large language models for low-code process automation: Lowering the barriers for citizen developers [Paper 242]. *ACIS 2025 Proceedings*. https://aisel.aisnet.org/acis2025/242/

Friel, A. (2025). *Using PLANS.md for multi-hour problem solving* [OpenAI Cookbook guidance on ExecPlans / living plan documents]. OpenAI. Retrieved October 29, 2026, from https://developers.openai.com/cookbook/articles/codex_exec_plans

Gadde, A. (2025). Democratizing software engineering through generative AI and vibe coding: The evolution of no-code development. *Journal of Computer Science and Technology Studies*, *7*(4), 556–572. https://doi.org/10.32996/jcsts.2025.7.4.66

Galster, M., Mohsenimofidi, S., Lulla, J. L., Abubakar, M. A., Treude, C., & Baltes, S. (2026). Harness engineering for agentic AI coding tools: An exploratory study [Version 5, revised 30 June 2026]. https://doi.org/10.48550/arXiv.2602.14690

Ge, Y., Mei, L., Duan, Z., Li, T., Zheng, Y., Wang, Y., Wang, L., Yao, J., Liu, T., Cai, Y., Bi, B., Guo, F., Guo, J., Liu, S., & Cheng, X. (2025). A survey of vibe coding with large language models. https://doi.org/10.48550/arXiv.2510.12399

Ginac, F. (n.d.). *Epistemological Debt: Cognitive Atrophy and Systemic Collapse in AI-Dependent Software Engineering* [Research manuscript; publication year and venue not established from opening text]. https://drive.google.com/file/d/1tNXR0gOJaU9BK-kQWaS6zLgus_XunCly/view?usp=drivesdk

Hassan, A. E., Li, H., Lin, D., Adams, B., Chen, T.-H., Kashiwa, Y., & Qiu, D. (2025). Agentic software engineering: Foundational pillars and a research roadmap. https://doi.org/10.48550/arXiv.2509.06216

Hassan, A. E., Oliva, G. A., Lin, D., Chen, B., & Jiang, Z. M. ((2024). Towards AI-native software engineering (SE 3.0): A vision and a challenge roadmap. https://doi.org/10.48550/arXiv.2410.06107

Hoda, R. (2025). Toward agentic software engineering beyond code: Framing vision, values, and vocabulary [Also appears in ICSE-Companion 2026]. https://doi.org/10.48550/arXiv.2510.19692

Hong, K., Troynikov, A., & Huber, J. (2025, July). *Context rot: How increasing input tokens impacts LLM performance* (tech. rep.). Chroma. Retrieved October 29, 2026, from https://research.trychroma.com/context-rot

Kang, R. (2026). Governed AI-assisted engineering: Graduated human oversight for agentic code generation in regulated domains. https://doi.org/10.48550/arXiv.2606.22484

Karpathy, A. (2025, February). There's a new kind of coding i call "vibe coding" [Accessed: 2026-09-17]. https://x.com/karpathy/status/1886192184808149383

Kobiella, C., Breidenstein, D., & Schmidt, A. (2026). From throw-away to takeaway: How GenAI and vibe coding accelerate prototyping across technical skill levels. *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, 1–23. https://doi.org/10.1145/3772318.3790757

Koch, C. (2026). Agentic Agile-V: From Vibe Coding to Verified Engineering in Software and Hardware Development [Stored arXiv version 1]. https://arxiv.org/abs/2605.20456v1

Koch, C., & Wellbrock, J. A. (2026). Agile V: A Compliance-Oriented Framework for AI-Augmented Engineering — From Concept to Audit-Ready Delivery [Stored arXiv version 1]. https://arxiv.org/abs/2602.20684v1

Li, H., Zhang, H., & Hassan, A. E. (2025). The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering [Stored arXiv version 1]. https://arxiv.org/abs/2507.15003v1

Li, J., Hou, Y., Lin, L., Zhu, R., Cao, H., & El Ali, A. (2026). Vibe Coding in Product Teams: Reconfiguring AI-Assisted Workflows, Prototyping, and Collaboration [Stored arXiv version 3; Conference named in the draft; placeholder DOI omitted]. *Proceedings of the 5th Annual Symposium on Human-Computer Interaction for Work*. https://arxiv.org/abs/2509.10652v3

Lopopolo, R. (2026, February 11). *Harness engineering: Leveraging Codex in an agent-first world*. OpenAI. Retrieved October 22, 2026, from https://openai.com/index/harness-engineering/

Maes, S. H. (2026). Vibe-coding and SDLC constrained and managed by an application-aware AI-like agentic platform. https://doi.org/10.5281/zenodo.18972674

Malterud, K., Siersma, V. D., & Guassora, A. D. (2016). Sample size in qualitative interview studies: Guided by information power. *Qualitative Health Research*, *26*(13), 1753–1760. https://doi.org/10.1177/1049732315617444

Mandl, P., & Mandl, P. (2026). AI-driven software development: A pragmatic path to agentic development processes. https://doi.org/10.48550/arXiv.2606.15283

Meske, C., Hermanns, T., von der Weiden, E., Loser, K.-U., & Berger, T. (2025). Vibe coding as a reconfiguration of intent mediation in software development: Definition, implications, and research agenda. *IEEE Access*, *13*, 213242–213259. https://doi.org/10.1109/ACCESS.2025.3645466

Mitchell, M., Ghosh, A., & Passi, S. (2026). AI Agents Push Humans Out of the Loop [Stored arXiv version 3]. https://arxiv.org/abs/2608.23642v3

Moreira, J. (2026). IACDM: Interactive Adversarial Convergence Development Methodology [Stored arXiv version 3; Technical foundation document / methodological preprint]. https://arxiv.org/abs/2604.16399v3

Osmani, A. (2024, December). *The 70% problem: Hard truths about AI-assisted coding*. Retrieved October 29, 2026, from https://addyo.substack.com/p/the-70-problem-hard-truths-about

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, *24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Režun, T. (2026). *Chasing jarvis: Can technically sophisticated non-programmers deploy SaaS applications using AI coding agents? an empirical investigation through context engineering and the lumina AI case study* [Author-posted manuscript, uploaded March 2026]. https://www.researchgate.net/publication/402083544

Sapkota, R., Roumeliotis, K. I., & Karkee, M. (2025). Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic AI. https://doi.org/10.48550/arXiv.2505.19443

Sarkar, A., & Drosos, I. (2025). Vibe coding: Programming through conversation with artificial intelligence. *Proceedings of the 36th Annual Conference of the Psychology of Programming Interest Group*. https://www.microsoft.com/en-us/research/publication/vibe-coding-programming-through-conversation-with-artificial-intelligence/

Sein, M. K., Henfridsson, O., Purao, S., Rossi, M., & Lindgren, R. (2011). Action design research. *MIS Quarterly*, *35*(1), 37–56. https://doi.org/10.2307/23043488
Storey, M.-A. (2026). From technical debt to cognitive and intent debt: Rethinking software health in the age of AI. https://doi.org/10.48550/arXiv.2603.22106
StrongDM AI. (2026, February 6). *Software factories and the agentic moment*. StrongDM. Retrieved October 23, 2026, from https://factory.strongdm.ai/
Uddin, S. M. J. (n.d.). *Is Vibe Coding the Future? An Empirical Assessment of LLM Generated Codes for Construction Safety* [Research manuscript; publication year and venue not established from opening text]. https://drive.google.com/file/d/19fGrczKwDzoFCQSpodmtuu6W0c1x3SZr/view?usp=drivesdk
Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: A framework for evaluation in design science research. *European Journal of Information Systems*, *25*(1), 77–89. https://doi.org/10.1057/ejis.2014.36
Viljoen, A., Radić, M., Hein, A., Nguyen, J., & Krcmar, H. (2024). Governing citizen development to address low-code platform challenges. *MIS Quarterly Executive*, *23*(3), 305–324. https://doi.org/10.17705/2msqe.00100
Wang, S. (2026). VibeContract: The Missing Quality Assurance Piece in Vibe Coding [Stored arXiv version 1; Placeholder 2024 date, conference name and DOI omitted]. https://arxiv.org/abs/2603.15691v1
Wang, Z., & Liu, M. (2026). Software engineering in the agent era: From trustworthy change to human–agent software organizations. https://doi.org/10.48550/arXiv.2609.04630
Waseem, M., Ahmad, A., Kemell, K.-K., Rasku, J., Lahti, S., Mäkelä, K., & Abrahamsson, P. (2025). Vibe coding in practice: Flow, technical debt, and guidelines for sustainable use. https://doi.org/10.48550/arXiv.2512.11922

A Testing across the development levels

This appendix summarises how testing is divided between the three levels of the framework: the self-checks of the Build Agent within SCOPE-V, the independent tests of a change within Agile V, and the tests of the whole application in stage 4. The division follows three rules. First, each level answers one question. Second, each test is written once, at one level; every later level runs all earlier tests again automatically. Third, each level adds only what the level before could not check: Agile V adds independence, because its tests are written by an agent that has not seen the code, and stage 4 adds reality, because it tests in a real environment and with real users. Table 19 shows who does what and when. Table 20 lists all tests and scans and shows at which level each one is written and where it is run again.

Table 19. Who does what and when at the three testing levels (own representation).

	SCOPE-V (Build Agent self-checks)	**Agile V** (Checkpoint B)	**Test application** (stage 4, gate G4)
Question	Does the **task** work?	Does the **change** do what the specification says?	Is the **whole application** ready for real users?
When	During each task	After all tasks of one change are complete	Once after gate G3
Environment	Development sandbox	Development sandbox; other systems may be simulated (mocks)	Staging: a copy of the production set-up with real login, real roles and test connections to other systems
New tests written here	Unit tests; quick code checks	Acceptance, contract, negative and boundary, and integration tests; full scans of the change	Smoke test, end-to-end journeys, real integration, full role matrix, security tests of the running application, data protection check, user acceptance test, operational readiness
Tests run again	None	All earlier tests (regression)	All tests from SCOPE-V and Agile V, now on staging
Written by	Build Agent	Test Designer Agent, without access to the code; IT sets the scanning rules	Users and business owner write test cards; the Test Designer Agent turns them into browser tests; IT and specialists define the expert checks
Run by	Build Agent	Verification Agent and automated pipeline	Verification Agent and pipeline; users; IT and specialists
Decision	None; the results do not count as evidence	Checkpoint B: business acceptance by the citizen developer and technical approval by IT	Gate G4: decision-makers according to the risk tier
If a test fails	The Build Agent repairs the code	The failure report goes back to the Build Agent; after a limited number of repair rounds, the citizen developer or IT takes over	Depends on whether the application did what the specification says. If not, the code is wrong: new change through the Agile V cycle. If it did, the specification is wrong: back to Checkpoint A

Table 20. All tests and scans and the level at which they are used (own representation).

Test or scan	What it checks	SCOPE-V	Agile V	Test application
Code checks				
Code-form checks	Obvious mistakes such as misspelled names or values of the wrong kind (linting, type checks)	New, one task	Run again, whole change	Run again, whole application
Static analysis	Known unsafe patterns, found by reading the code without running it	Quick version	Full, whole change	Run again, whole application
Secret scanning	Passwords or access keys written into the code	Quick version	Full, whole change	Run again, whole application
Dependency scan	Known security problems in external libraries	–	New	Run again
Library allow-list check	New external libraries are on the list approved by IT	–	New	Run again
AI code review	Advisory comments on security, errors and maintainability	–	New	–
Functional tests				
Unit tests	The smallest parts of the code	New	Run again	Run again
Acceptance tests	Each acceptance criterion of the specification, including role rules; the main tests of a change	–	New	Run again
Contract tests	Rules that must always hold, for example that a balance never becomes negative	–	New	Run again
Negative and boundary tests	Wrong inputs, limits and forbidden actions	–	New	Run again
Integration tests	Whether the change works with neighbouring parts; other systems may be simulated	–	New	Run again
Regression run	Whether existing functions still work, by running all earlier tests again	–	All earlier tests	All tests, on staging
Tests of the whole application				
Smoke test	Whether the application starts, login works and connected systems respond	–	–	New
Real integration	The real database, login and Workday test connection instead of simulated systems	–	–	New
End-to-end journeys	Complete user tasks across several features, run in the browser from the users' test cards	–	–	New
Full role matrix	All roles and all actions: permitted actions work, forbidden actions are blocked	–	–	New
Security and data protection of the whole application				
Dynamic security scan	A tool attacks the running application from outside	–	–	New (R2 and higher)
Security test agent	An agent attempts forbidden actions from the perspective of each role on the running application	–	–	New (R2 and higher)
Penetration test	A security expert tries to break into the application	–	–	New (R3 only)
Data protection check	Which personal data are stored, logged and deleted, and where	–	–	New (if triggered)
Readiness for users and operation				
User acceptance test	Real users complete the test cards without help from the citizen developer	–	–	New
Operational readiness	Rollback, monitoring, backup and a named operations owner	–	–	New

Note. New: the test is written at this level. Run again: an existing test is repeated. Quick version and Full: the same check with a narrower or wider scope. –: not done at this level.